

Gorgo Go for Java Programmers — Answer Key

May 31, 2026

About This Answer Key

Work out the answers yourself before looking here. The explanations are meant to help you understand *why* the answer is what it is, not just tell you *what* the answer is.

Chapter 0: How to Use This Booklet

There are no exercises in Chapter 0.

Chapter 1: Hello, Go

Exercise 1 (Think about it): Java has four visibility levels: `public`, `protected`, package-private (no keyword), and `private`. Go has two: exported (uppercase) and unexported (lowercase), with the package as the only boundary. What do you gain from Go’s simpler model? What do you lose? Can you think of a Java visibility pattern that has no direct equivalent in Go?

What you gain:

- Simplicity — one rule covers everything: uppercase = visible outside the package, lowercase = not. There is no keyword to remember, no risk of accidentally using `protected` when you meant `private`, and no six-word modifier like `public static final`.
- Consistency — the rule applies uniformly to functions, types, variables, struct fields, and methods. There is no special case for classes vs members.
- Readability — any identifier starting with a capital letter is part of the public API; the rest is implementation detail. You can tell at a glance what is exported without an IDE.

What you lose:

- Granularity — Java’s four levels let you express “visible to subclasses in other packages” (`protected`) and “visible within the package only” (package-private) as distinct states. Go collapses both into “unexported.”
- Per-symbol control within a package — every unexported name in a package is equally visible to every other file in that package. You cannot hide a helper from one file while exposing it to another in the same package.

A Java pattern with no direct Go equivalent:

`protected` members — visible to subclasses anywhere but hidden from unrelated code in other packages. Go has no inheritance, so the concept of “subclass visibility” does not exist. The closest substitutes are the `internal/` directory convention (covered in Chapter 13) and interfaces, but neither maps exactly to `protected`.

Exercise 2 (What does this print?):

```
package main

import "fmt"

func main() {
    name := "Chappell Roan"
    plays := 1_500_000
    fmt.Printf("%s has %d plays\n", name, plays)
    fmt.Printf("type of plays: %T\n", plays)
    fmt.Println(fmt.Sprintf("quoted: %q", name))
}
```

Output:

```
Chappell Roan has 1500000 plays
type of plays: int
quoted: "Chappell Roan"
```

`%s` formats the string without quotes. `%d` formats the integer in base 10; the `_` digit separator in the source literal `1_500_000` is purely cosmetic — the value is 1500000. `%T` prints the Go type name, which for an integer literal assigned with `:=` is `int`. `%q` wraps the string in double quotes and escapes any characters that need it. `fmt.Sprintf` returns the formatted string; `fmt.Println` then prints it with a newline appended.

Exercise 3 (Calculation): You run the following program as:

```
go run main.go Espresso Gresso Sabrina
```

```
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Println(len(os.Args))
    fmt.Println(os.Args[2])
}
```

What does it print?

`os.Args` contains every token on the command line including the program name at index 0. The full slice is `["<binary>", "Espresso", "Gresso", "Sabrina"]`, so `len(os.Args)` is 4. `os.Args[2]` is "Gresso" (the second user-supplied argument, at index 2).

Output:

```
4
Gresso
```

Exercise 4 (Where is the bug?):

```
package main
```

```
import (
    "fmt"
    "math"
)

func main() {
    fmt.Println("Hello, Go!")
}
```

"math" is imported but never used. The program will not compile. The compiler produces:

```
./main.go:5:2: "math" imported and not used
```

The fix is to remove the "math" import. If you were writing this in an editor with `goimports` configured to run on save, it would have removed the unused import automatically before you even tried to build.

Exercise 5 (Write a program): Write a Go program that accepts a song title as the first command-line argument and a play count as the second, then prints them formatted as "<title> has <count> plays". If fewer than two arguments are provided (not counting the program name), print a usage message and exit.

```
package main

import (
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 3 {
        fmt.Fprintln(os.Stderr, "usage: plays <title> <count>")
        os.Exit(1)
    }
    title := os.Args[1]
    count := os.Args[2]
    fmt.Printf("%q has %s plays\n", title, count)
}
```

Sample run:

```
$ go run main.go "Espresso" 1500000
"Espresso" has 1500000 plays
```

Notes: - `len(os.Args) < 3` because index 0 is the binary name, 1 is the title, and 2 is the count. - The play count is kept as a string here; Chapter 3 covers `strconv.Atoi` for converting it to an integer. - `fmt.Fprintln(os.Stderr, ...)` sends the usage message to standard error, which is the convention for diagnostic output. - `os.Exit(1)` terminates the program immediately with a non-zero exit code, signalling failure to the shell.

Chapter 2: Types and Variables

Exercise 1 (Think about it): In Java, using an uninitialized local variable is a compile error. In Go, every variable has a zero value. What are the benefits and potential risks of zero values?

The primary benefit is safety and predictability: there are no uninitialized reads and no undefined behavior from reading a garbage value off the stack. You can declare a `sync.Mutex` or a `bytes.Buffer` and use it

immediately without calling a constructor, because the zero value is deliberately designed to be a valid, ready-to-use state. This is a Go idiom worth internalizing: if you design a type so that its zero value is useful, callers pay no initialization tax.

The risk is silent logic errors. In Java, forgetting to initialize a variable is a compile-time catch. In Go, `var count int` silently starts at 0, `var name string` at "", and `var ptr *SomeType` at `nil`. If you forget to set `ptr` before dereferencing it, you get a runtime panic, not a compile error. Similarly, a `bool` zero value is `false`, so a field like `IsAdmin` starts as `false` — which is the safe default — but a field like `IsEnabled` also starts as `false`, which might not be what you want. Zero values encourage a different discipline: design your data so that the zero state is meaningful and correct.

Exercise 2 (What does this print?):

```
package main

import "fmt"

type StreamingTier int

const (
    Free StreamingTier = iota
    Standard
    Premium
    Lossless
)

func main() {
    fmt.Println(Free, Standard, Premium, Lossless)
    fmt.Printf("Premium = %d\n", Premium)
}
```

Output:

```
0 1 2 3
Premium = 2
```

`iota` starts at 0 for the first constant in a `const` block and increments by 1 for each subsequent constant. `Free` gets 0, `Standard` gets 1, `Premium` gets 2, and `Lossless` gets 3. Because `StreamingTier` is a named type based on `int`, `%d` formats it as a plain integer. `fmt.Println` with multiple arguments separates them with spaces and appends a newline.

Exercise 3 (Calculation): What is the zero value of each of the following types: `int`, `float64`, `bool`, `string`, `*int`?

Type	Zero value
<code>int</code>	0
<code>float64</code>	0.0
<code>bool</code>	<code>false</code>
<code>string</code>	"" (empty string, length 0)
<code>*int</code>	<code>nil</code>

Every variable declared with `var` and no initializer gets its type's zero value. `nil` is the zero value for all pointer types, as well as slices, maps, channels, functions, and interfaces. A `nil` pointer is safe to declare but will panic if you dereference it.

Exercise 4 (Where is the bug?):

```
package main

import "fmt"

func main() {
    x := 10
    x := 20
    fmt.Println(x)
}
```

`:=` requires at least one new variable on the left side. Because `x` is already declared in the same scope, the second `:=` is a compile error:

```
./main.go:7:4: no new variables on left side of :=
```

The fix depends on intent. To reassign `x`, use a plain `=`:

```
x := 10
x = 20
fmt.Println(x)
```

`:=` is valid the second time only if you are introducing at least one new variable alongside the existing one, for example `x, y := 20, 30`.

Exercise 5 (Write a program): Define types `Celsius` and `Fahrenheit` based on `float64`. Write conversion functions. Print the boiling and freezing points of water in both scales.

```
package main

import "fmt"

type Celsius float64
type Fahrenheit float64

func CToF(c Celsius) Fahrenheit {
    return Fahrenheit(c*9/5 + 32)
}

func FToC(f Fahrenheit) Celsius {
    return Celsius((f - 32) * 5 / 9)
}

func main() {
    freezingC := Celsius(0)
    boilingC := Celsius(100)

    fmt.Printf("Freezing: %.1f°C = %.1f°F\n", freezingC, CToF(freezingC))
    fmt.Printf("Boiling: %.1f°C = %.1f°F\n", boilingC, CToF(boilingC))

    freezingF := Fahrenheit(32)
    boilingF := Fahrenheit(212)

    fmt.Printf("Freezing: %.1f°F = %.1f°C\n", freezingF, FToC(freezingF))
}
```

```
    fmt.Printf("Boiling: %.1f°F = %.1f°C\n", boilingF, FToC(boilingF))
}
```

Output:

```
Freezing: 0.0°C = 32.0°F
Boiling: 100.0°C = 212.0°F
Freezing: 32.0°F = 0.0°C
Boiling: 212.0°F = 100.0°C
```

The key insight is that `Celsius` and `Fahrenheit` are distinct types even though both are backed by `float64`. You cannot pass a `Celsius` where a `Fahrenheit` is expected without an explicit conversion. This is why type definitions exist: they let the compiler catch unit errors at compile time rather than at runtime.

Chapter 3: Strings, Bytes, and Runes

Exercise 1 (Think about it): If Go strings are byte sequences and not character sequences, what happens when you index into a string containing a multibyte character like `é`? How does `for range` behave differently from `for i := 0; i < len(s); i++`?

Indexing with `s[i]` yields the byte at position `i`, not the character. For ASCII, one byte is one character, so `s[0]` on `"café"` gives `'c'` as a byte (`uint8` value 99). But `é` in UTF-8 is encoded as two bytes (`0xC3 0xA9`). If `"café"` starts at index 0, then `s[3]` is `0xC3` — the first byte of `é` — not the character `é` itself. This is often surprising and is a common source of bugs when programmers index into strings that may contain non-ASCII characters.

`for i := 0; i < len(s); i++` walks byte by byte, so iterating `"café"` visits 5 bytes (`c`, `a`, `f`, and the two bytes of `é`).

`for i, r := range s` decodes the string as UTF-8 on each iteration. `i` is the byte offset of the start of the rune, and `r` is the decoded **rune** (a full Unicode code point as `int32`). Iterating `"café"` with `range` visits 4 runes: `'c'`, `'a'`, `'f'`, and `'é'`. When `range` reaches the two-byte sequence for `é`, it decodes both bytes together and advances `i` by 2. Use `range` when you care about characters; use byte indexing only when you are sure the string is pure ASCII or when you genuinely need to operate on raw bytes.

Exercise 2 (What does this print?):

```
package main

import "fmt"

func main() {
    s := "café"
    fmt.Println("len:", len(s))
    fmt.Printf("s[3] = %d (0x%X)\n", s[3], s[3])

    for i, r := range s {
        fmt.Printf("index %d: %c (%d)\n", i, r, r)
    }
}
```

Output:

```
len: 5
s[3] = 195 (0xC3)
```

```
index 0: c (99)
index 1: a (97)
index 2: f (102)
index 3: é (233)
```

"café" is 5 bytes long: c a f are one byte each, and é is two bytes (0xC3 0xA9). `s[3]` returns the byte at position 3, which is 0xC3 (decimal 195) — the first byte of the UTF-8 encoding of é, not the character itself. `range` decodes the UTF-8 properly: it sees the two bytes at position 3 as a single rune (é, code point U+00E9, decimal 233) and skips byte 4 entirely. Notice that index 4 never appears in the range output because é occupies two byte positions (3 and 4) but is a single rune.

Exercise 3 (Calculation): How many bytes does `len("Beyoncé")` return? How many runes does `utf8.RuneCountInString("Beyoncé")` return?

`len("Beyoncé")` returns 8. B, e, y, o, n, c are each one byte (6 bytes total), and é (U+00E9) encodes to two bytes in UTF-8, giving 8 bytes.

`utf8.RuneCountInString("Beyoncé")` returns 7. The string contains 7 characters: B, e, y, o, n, c, and é. `len` counts bytes; `utf8.RuneCountInString` counts Unicode code points. They agree on pure ASCII strings and diverge the moment any character requires more than one byte.

Exercise 4 (Where is the bug?):

```
package main

import "fmt"

func shout(s string) string {
    result := make([]byte, len(s))
    for i := 0; i < len(s); i++ {
        result[i] = s[i] - 32
    }
    return string(result)
}

func main() {
    fmt.Println(shout("café"))
}
```

The function assumes every character is a single byte and that subtracting 32 uppercases it. Both assumptions are wrong for non-ASCII input.

For ASCII letters, subtracting 32 from a lowercase byte gives the corresponding uppercase byte (e.g., 'a' - 32 = 'A'). But "café" is 5 bytes, and bytes 3 and 4 are the UTF-8 encoding of é (0xC3 and 0xA9). Subtracting 32 from each gives 0xA3 and 0x89, which are not a valid UTF-8 sequence for any meaningful character. The result is garbled output or a replacement character, not "CAFÉ".

The correct approach for Unicode-aware uppercasing is `strings.ToUpper`:

```
import "strings"

func shout(s string) string {
    return strings.ToUpper(s)
}
```

Output:

Exercise 5 (Write a program): Write a program that reverses a string correctly — by rune, not by byte — and prints the result.

```
package main

import "fmt"

func reverseString(s string) string {
    runes := []rune(s)
    for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
        runes[i], runes[j] = runes[j], runes[i]
    }
    return string(runes)
}

func main() {
    words := []string{"café", "Beyoncé", "hello"}
    for _, w := range words {
        fmt.Printf("%q -> %q\n", w, reverseString(w))
    }
}
```

Output:

```
"café" -> "éfac"
"Beyoncé" -> "écnoyeb"
"hello" -> "olleh"
```

The key step is converting the string to `[]rune` first. This decodes the UTF-8 and gives you one `rune` per Unicode code point regardless of how many bytes each one occupies. You then swap elements in the rune slice in place, and convert back to a `string` at the end. Reversing the raw `[]byte` instead would shuffle the individual bytes of multibyte characters and produce invalid UTF-8.

Chapter 4: Control Flow

Exercise 1 (Think about it): Go has only one loop keyword, `for`. Java has `for`, `while`, and `do...while`. Is this limiting, or does it simplify the language? Can you think of a pattern where a `do...while` loop cannot be elegantly expressed with Go's `for`?

Having one loop keyword is not limiting in practice. Go's `for` covers all three Java forms:

- C-style `for`: `for i := 0; i < n; i++ { ... }`
- while-style: `for condition { ... }`
- infinite: `for { ... }` with a `break` inside

The `do...while` pattern — execute the body at least once, then check the condition — requires a small idiom in Go:

```
for {
    // body
    if !condition {
        break
    }
}
```



```
}  
}
```

This is slightly less elegant than Java's `do { ... } while (condition)` because the loop-exit logic is inside the body rather than at the bottom of the statement. For readers scanning code quickly, the exit condition is less visible. In practice, `do...while` is rare enough in both languages that this is a minor inconvenience. The benefit is a smaller language: one keyword to teach, one loop construct to remember, and fewer edge cases around scoping and control flow.

Exercise 2 (What does this print?):

```
package main  
  
import "fmt"  
  
func main() {  
    for i := 0; i < 3; i++ {  
        defer fmt.Println(i)  
    }  
    fmt.Println("done")  
}
```

Output:

```
done  
2  
1  
0
```

Two things are happening here. First, `defer` runs after the surrounding function returns, so all three deferred calls happen after `fmt.Println("done")`. Second, `defer` arguments are evaluated immediately at the point of the `defer` statement, not when the deferred call executes. When `i` is 0, `defer fmt.Println(0)` is registered with the value 0 baked in. When `i` is 1, `defer fmt.Println(1)` is registered with 1. When `i` is 2, `defer fmt.Println(2)` is registered with 2. Deferred calls execute in LIFO (last-in, first-out) order, so the last one registered runs first: 2, then 1, then 0.

Exercise 3 (What does this print?):

```
package main  
  
import "fmt"  
  
func grade(score int) string {  
    switch {  
    case score >= 90:  
        return "A"  
    case score >= 80:  
        return "B"  
    case score >= 70:  
        return "C"  
    default:  
        return "F"  
    }  
}
```

```
func main() {
    fmt.Println(grade(95))
    fmt.Println(grade(83))
    fmt.Println(grade(70))
    fmt.Println(grade(55))
}
```

Output:

```
A
B
C
F
```

An expression-less `switch` is equivalent to `switch true` — each case is a boolean expression, and the first one that evaluates to `true` wins. Cases are evaluated top to bottom; once a match is found, the remaining cases are skipped. There is no fallthrough, so `grade(83)` matches `score >= 80`, returns "B", and never reaches `score >= 70`. The `default` clause matches when no case is true.

Exercise 4 (Where is the bug?):

```
package main

import "fmt"

func makeMultipliers() []func(int) int {
    fns := make([]func(int) int, 3)
    factor := 1
    for i := 0; i < 3; i++ {
        factor = (i + 1) * 10
        fns[i] = func(x int) int { return x * factor }
    }
    return fns
}

func main() {
    fns := makeMultipliers()
    for _, f := range fns {
        fmt.Println(f(5))
    }
}
```

All three calls print 150, not 50, 100, 150.

The bug is that `factor` is declared outside the loop. All three closures capture the same `factor` variable by reference. By the time the closures run, the loop has finished and `factor` is 30 (the last value assigned). Every closure multiplies by 30, so `f(5)` returns 150 for all three.

The fix is to declare `factor` inside the loop so each iteration gets its own copy:

```
for i := 0; i < 3; i++ {
    factor := (i + 1) * 10
    fns[i] = func(x int) int { return x * factor }
}
```

Now each closure captures a distinct `factor` variable, and the output is 50, 100, 150.

Exercise 5 (Write a program): Write a program using `defer` and `recover` to catch a panic from a function that calls `panic("something went wrong")`, print the recovered message, and continue execution normally.

```
package main

import "fmt"

func safeRun(f func()) {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("recovered:", r)
        }
    }()
    f()
}

func riskyOperation() {
    fmt.Println("starting risky operation")
    panic("something went wrong")
}

func main() {
    fmt.Println("before safeRun")
    safeRun(riskyOperation)
    fmt.Println("after safeRun --- execution continues normally")
}
```

Output:

```
before safeRun
starting risky operation
recovered: something went wrong
after safeRun --- execution continues normally
```

`recover` only works when called directly inside a deferred function. The deferred anonymous function runs when `riskyOperation` panics; at that point, `recover()` returns the value passed to `panic` and stops the panic from propagating up the call stack. After `safeRun` returns normally, `main` continues.

Note that `panic` and `recover` are not Go's primary error handling mechanism — that role belongs to returning `error` values. Use `panic/recover` only for truly unrecoverable situations or at the boundary of a library to prevent a panic from escaping into caller code.

Chapter 5: Functions — Answers

Exercise 1 (Think about it): Go returns errors as values rather than throwing exceptions. A Java checked exception forces the caller to handle it — the compiler will not let you ignore it. Go's multi-return error is also explicit, but you can discard it with `_` or simply not assign the second return value. Does Go's approach give you the same safety guarantee as Java's checked exceptions? What is gained and what is lost by each approach?

Java's checked exceptions provide a compiler-enforced contract: if a method declares `throws IOException`, the caller must either catch it or declare that it also throws it. There is no way to silently ignore a checked exception in Java without at least writing a catch block (even an empty one is conspicuous).

Go does not provide that same guarantee. You can write `result, _ := divide(a, b)` or even `result :=`

`somePackage.Lookup(key)` if you know (or ignore) that `Lookup` returns `(string, error)` but only assign one variable — though the compiler will reject an assignment that captures the wrong number of values, it will happily accept `_` for any of them.

What Go gains:

- Error handling is explicit code, not a separate control-flow mechanism. Errors flow through the same call stack as regular values, which makes them easier to wrap, annotate, and inspect.
- There is no distinction between checked and unchecked exceptions to manage. In Java, many APIs force you to wrap checked exceptions in `RuntimeException` just to use them in lambdas or streams.
- Errors are plain values you can store, compare, pass around, and test — no reflection required.

What Java gains:

- The compiler can prove at build time that every failure path is addressed. Go relies on code review and tools like `errcheck` to catch ignored errors.
- Stack traces are attached to exceptions automatically; in Go you must explicitly wrap errors with `fmt.Errorf("...: %w", err)` to build a chain.

In practice, Go's approach leads to more explicit error-handling code and fewer “surprise” failures from unchecked exceptions — but it also produces more repetitive `if err != nil` checks that disciplined engineers must not skip.

Exercise 2 (What does this print?):

```
package main

import "fmt"

func makeAdder(n int) func(int) int {
    return func(x int) int {
        return x + n
    }
}

func main() {
    add5 := makeAdder(5)
    add10 := makeAdder(10)
    fmt.Println(add5(3))
    fmt.Println(add10(3))
    fmt.Println(add5(add10(1)))
}
```

Output:

```
8
13
16
```

`makeAdder(5)` returns a closure that captures `n = 5`. `makeAdder(10)` returns a separate closure that captures `n = 10`. Each closure has its own independent copy of `n` because each call to `makeAdder` creates a new variable.

`add5(3)` returns `3 + 5 = 8`. `add10(3)` returns `3 + 10 = 13`. `add5(add10(1))` evaluates inside-out: `add10(1)` returns `1 + 10 = 11`, then `add5(11)` returns `11 + 5 = 16`.

Exercise 3 (Calculation): Given the function below, what values are printed by the three `fmt.Println` calls? Trace the value of `total` at each step.

```
package main

import "fmt"

func running(start int) func(int) int {
    total := start
    return func(n int) int {
        total += n
        return total
    }
}

func main() {
    acc := running(100)
    fmt.Println(acc(10))
    fmt.Println(acc(20))
    fmt.Println(acc(-5))
}
```

Output:

```
110
130
125
```

`running(100)` creates a closure that captures `total`, initialised to 100. The same `total` variable is shared across all calls through `acc` because the closure captures it by reference — it is the same memory location every time.

- `acc(10)`: `total = 100 + 10 = 110`; returns and prints 110.
- `acc(20)`: `total = 110 + 20 = 130`; returns and prints 130.
- `acc(-5)`: `total = 130 + (-5) = 125`; returns and prints 125.

This is a running total (accumulator) implemented with a closure. Each call modifies and returns the accumulated value.

Exercise 4 (Where is the bug?): The following code tries to build a slice of greeting functions, one for each name in a list, using a Go 1.21 module.

```
package main

import "fmt"

func main() {
    names := []string{"benenson", "amara", "priya"}
    greets := make([]func(), len(names))
    for i, name := range names {
        greets[i] = func() { fmt.Println("hola,", name) }
    }
    for _, g := range greets {
        g()
    }
}
```

The bug: Under Go 1.21 semantics, the `for range` loop reuses the same `i` and `name` variables for every iteration. All three closures capture the same `name` variable — not a copy of its value, but a reference to the single loop variable. By the time any of the closures run, the loop has finished and `name` holds the last value: `"priya"`.

Actual output under Go 1.21:

```
hola, priya
hola, priya
hola, priya
```

Expected output:

```
hola, benson
hola, amara
hola, priya
```

Fix 1 (works in all Go versions): Create a new local variable inside the loop body to shadow the loop variable:

```
for i, name := range names {
    name := name // create a new variable for this iteration
    greets[i] = func() { fmt.Println("hola,", name) }
}
```

Fix 2 (works in Go 1.22+): Update the `go` directive in `go.mod` to `go 1.22` or later. The language change makes each loop iteration create its own variable automatically, so all existing closure-in-loop code behaves correctly without any source changes.

Exercise 5 (Write a program): Write a function `pipeline(fns ...func(int) int) func(int) int` that takes any number of `func(int) int` functions and returns a new function that applies them in order.

```
package main

import "fmt"

// pipeline returns a function that applies each fn in fns left to right.
func pipeline(fns ...func(int) int) func(int) int {
    return func(x int) int {
        result := x
        for _, fn := range fns {
            result = fn(result) // apply each transform in sequence
        }
        return result
    }
}

func main() {
    double := func(n int) int { return n * 2 } // multiply by 2
    addTen  := func(n int) int { return n + 10 } // add 10
    square  := func(n int) int { return n * n } // square the value

    // double then addTen: (3*2)+10 = 16
    p1 := pipeline(double, addTen)
    fmt.Println(p1(3)) // 16

    // addTen then double: (3+10)*2 = 26
}
```

```

p2 := pipeline(addTen, double)
fmt.Println(p2(3)) // 26

// double then square: (3*2)^2 = 36
p3 := pipeline(double, square)
fmt.Println(p3(3)) // 36

// empty pipeline --- identity
p4 := pipeline()
fmt.Println(p4(7)) // 7
}

```

Key points in this solution:

- `pipeline` is variadic: it accepts any number of `func(int) int` values.
- Inside the returned closure, `fns` is captured by reference — the same `[]func(int) int` slice the outer call built.
- The order of application matters: `pipeline(double, addTen)` and `pipeline(addTen, double)` produce different results for the same input.
- An empty `pipeline()` call returns an identity function because the loop body never executes.

Chapter 6: Methods and Embedding — Answers

Exercise 1 (Think about it): In Java, a class bundles data and behavior together and inheritance lets you share both across a type hierarchy. Go separates data (struct), behavior (methods), and code reuse (embedding) into three distinct mechanisms, and interfaces handle polymorphism independently of all three. What advantages does Go’s separated approach offer over Java’s unified class model? Can you think of a scenario where Java’s approach is simpler or more convenient?

Advantages of Go’s separated approach:

1. **You can attach methods to any type, not just classes.** In Java, methods live inside class bodies and you cannot add methods to types defined in other packages. In Go, you can define methods on any named type in the same package, including types imported from the standard library via type definitions (e.g., `type Seconds float64`).
2. **Code reuse without coupling.** Java inheritance forces an is-a relationship: `FeaturedTrack extends Track` means every `FeaturedTrack` is substitutable for a `Track`. Go embedding is a has-a relationship with no substitutability. You get promoted fields and methods without locking yourself into a hierarchy that may become wrong later.
3. **Interfaces decouple behavior from data completely.** A type satisfies a Go interface without knowing the interface exists. This lets you define interfaces in the consumer package, not the producer package, making dependencies flow the right way.
4. **No fragile base-class problem.** Java’s virtual dispatch means a change to a superclass method can silently alter the behavior of all subclasses. Go’s promoted methods are not virtual: calling a promoted method on an outer struct always calls the embedded type’s method, unless the outer struct explicitly defines its own method with the same name.

Where Java’s approach is simpler:

- When you genuinely want polymorphism through a type hierarchy (e.g., a UI widget tree), Java’s `extends` gives you substitutability, virtual dispatch, and `instanceof` checks in one declaration. In Go you need an interface plus embedding, and you must ensure both the outer and inner types implement the interface explicitly.

- A `toString()` override in Java is automatic through the `Object` base class. In Go, `fmt.Stringer` requires you to implement `String()` `string` on each type that wants custom formatting; there is no default.

Exercise 2 (What does this print?):

```
package main

import "fmt"

type Base struct {
    ID int
}

func (b Base) Describe() string {
    return fmt.Sprintf("Base ID=%d", b.ID)
}

type Widget struct {
    Base
    Color string
}

func main() {
    w := Widget{
        Base: Base{ID: 42},
        Color: "blue",
    }
    fmt.Println(w.ID)
    fmt.Println(w.Color)
    fmt.Println(w.Describe())
    fmt.Println(w.Base.Describe())
}
```

Output:

```
42
blue
Base ID=42
Base ID=42
```

`w.ID` is promoted from `w.Base.ID` — accessing `w.ID` and `w.Base.ID` reach the same field. `w.Color` is a direct field of `Widget`. `w.Describe()` calls the promoted `Base.Describe()` method, because `Widget` has no `Describe` method of its own. `w.Base.Describe()` calls the same method through the explicit path. Both calls produce identical output.

Exercise 3 (Calculation): Trace the following program by hand.

```
package main

import "fmt"

type Track struct {
    Title string
}
```



```

    Artist string
    BPM    int
}

func (t Track) String() string {
    return fmt.Sprintf("%s by %s", t.Title, t.Artist)
}

type FeaturedTrack struct {
    Track
    Feature string
}

func (ft FeaturedTrack) String() string {
    return ft.Track.String() + " ft. " + ft.Feature
}

func main() {
    t := Track{Title: "Golden Hour", Artist: "JVKE", BPM: 97}
    ft := FeaturedTrack{Track: t, Feature: "Rosalía"}

    fmt.Println(t.String())
    fmt.Println(ft.String())
    fmt.Println(ft.Track.String())
    fmt.Println(ft.BPM)
}

```

Output:

```

Golden Hour by JVKE
Golden Hour by JVKE ft. Rosalía
Golden Hour by JVKE
97

```

Step by step:

- `t.String()`: calls `Track.String()` directly on `t`. Returns "Golden Hour by JVKE".
- `ft.String()`: `FeaturedTrack` has its own `String()` method, so the promoted `Track.String()` is shadowed. `FeaturedTrack.String()` calls `ft.Track.String()` (returns "Golden Hour by JVKE") and appends " ft. " and `ft.Feature`. Returns "Golden Hour by JVKE ft. Rosalía".
- `ft.Track.String()`: calls `Track.String()` through the explicit embedded path, bypassing `FeaturedTrack.String()`. Returns "Golden Hour by JVKE".
- `ft.BPM`: promoted from `ft.Track.BPM`. The value is 97.

The key insight: writing `ft.String()` calls `FeaturedTrack.String()` (the outer type's method), while `ft.Track.String()` bypasses the outer method and calls `Track.String()` directly.

Exercise 4 (Where is the bug?):

```

package main

import "fmt"

type Artist struct {
    Name string
}

```

```

func (a Artist) Label() string {
    return "Artist: " + a.Name
}

type Song struct {
    *Artist
    Title string
}

func main() {
    s := Song{Title: "Chorizo Asado"}
    fmt.Println(s.Title)
    fmt.Println(s.Label()) // line A
}

```

The bug: Song is initialized without setting the *Artist pointer, so s.Artist is nil. The first `fmt.Println(s.Title)` prints "Chorizo Asado" successfully. The second call `s.Label()` is promoted from the embedded *Artist. To call a method on the embedded type, Go dereferences the embedded pointer. Dereferencing a nil pointer causes a runtime panic:

panic: runtime error: invalid memory address or nil pointer dereference

Fix: initialize the embedded pointer before use:

```

s := Song{
    Artist: &Artist{Name: "Feid"},
    Title:  "Chorizo Asado",
}
fmt.Println(s.Title) // Chorizo Asado
fmt.Println(s.Label()) // Artist: Feid

```

Alternatively, construct Song using `NewSong` to enforce initialization:

```

func NewSong(title, artistName string) Song {
    return Song{Artist: &Artist{Name: artistName}, Title: title}
}

```

Exercise 5 (Write a program):

```
package main
```

```
import "fmt"
```

```
type Counter struct {
    Value int
}
```

```
func NewCounter(start int) *Counter { // constructor: returns *Counter so callers can use pointer rec
    return &Counter{Value: start}
}
```

```
func (c *Counter) Increment() { // pointer receiver: mutates Value
    c.Value++
}
```

```

func (c *Counter) Reset() {                                // pointer receiver: mutates Value
    c.Value = 0
}

func (c *Counter) String() string {                       // pointer receiver for consistency
    return fmt.Sprintf("count: %d", c.Value)
}

func main() {
    c := NewCounter(10)
    c.Increment()
    c.Increment()
    c.Increment()
    fmt.Println(c)    // count: 13
    c.Reset()
    fmt.Println(c)    // count: 0
}

```

Output:

```

count: 13
count: 0

```

Notes: - `NewCounter` returns `*Counter` so every method call works without taking an address at the call site. - All three methods use pointer receivers for consistency — since `Increment` and `Reset` must mutate `c.Value`, all methods on `*Counter` use the pointer form. - `fmt.Println(c)` calls `c.String()` automatically because `*Counter` satisfies `fmt.Stringer` (which requires `String() string`).

Chapter 7: Maps and Slices — Answers

Exercise 1 (Think about it): In Java, `HashMap<K,V>` requires keys to implement `hashCode()` and `equals()`, and `ArrayList<E>` stores references to boxed objects on the heap. Go's `map[K]V` requires `K` to be comparable at the language level, and a `[]E` slice stores values directly in the backing array. What are the trade-offs of Go's approach for each collection type? Give one example of a Java key type you cannot use directly as a Go map key, and explain one scenario where storing values directly in a slice (rather than as heap references) matters for performance. When would you feel the difference most?

A Java `ArrayList<Integer>` stores a pointer (reference) to each boxed `Integer` object, and each `Integer` object lives somewhere on the heap. Iterating over the list means following a pointer for every element, and those `Integer` objects may be scattered around memory — poor cache locality. Each `Integer` also carries object header overhead (typically 16 bytes) even though the actual integer value is just 4 bytes.

A Go `[]int` stores the integer values **directly** and **contiguously** in the backing array. Iterating is a sequential scan through a flat region of memory: the CPU prefetcher handles this extremely well. There is no per-element allocation overhead and no pointer chasing.

You feel the difference most in:

- **Tight loops** that process large slices: the sequential memory access pattern is cache-friendly, and modern CPUs can vectorize flat integer arrays.
- **Memory usage**: a Go `[]int` of a million elements is roughly 8 MB (64-bit ints). A Java `ArrayList<Integer>` of a million elements is the list's pointer array (8 MB of references) plus a million `Integer` objects on the heap (at least 16 MB more), for a minimum of 24 MB — and GC pressure from all those small objects.

- **GC pauses:** the Go garbage collector has no pointers to trace inside a `[]int`, so it scans the array in constant time. A Java `ArrayList<Integer>` forces the GC to follow a million references.

The trade-off is that Go's approach works for slices of value types (`int`, `float64`, structs). For slices of interfaces or pointers, Go has the same indirection that Java does.

Exercise 2 (What does this print?):

```
package main

import "fmt"

func main() {
    catalog := map[string]int{
        "Blinding Lights": 4_000_000_000,
        "Shape of You":    3_600_000_000,
    }
    hits := []string{"Shape of You", "Watermelon Sugar", "Blinding Lights"}
    for _, title := range hits {
        if plays, ok := catalog[title]; ok {
            fmt.Printf("%s: %d\n", title, plays)
        } else {
            fmt.Printf("%s: not found\n", title)
        }
    }
}
```

Output:

```
Shape of You: 3600000000
Watermelon Sugar: not found
Blinding Lights: 4000000000
```

The loop iterates the `hits` slice in order. "Shape of You" is in the catalog and its play count is printed. "Watermelon Sugar" is not in the catalog, so the comma-ok idiom sets `ok = false` and the `else` branch runs. "Blinding Lights" is in the catalog and is printed last. Map lookup order is random, but slice range iteration is always in index order, so the output is deterministic here.

Exercise 3 (Calculation): Given the following code, trace the value of `len(s)` and `cap(s)` after each line.

```
s := make([]int, 2, 5)
s = append(s, 10)
s = append(s, 20)
s = append(s, 30)
s = append(s, 40)
```

After line	len(s)	cap(s)	New array?
<code>make([]int, 2, 5)</code>	2	5	Yes (initial)
<code>append(s, 10)</code>	3	5	No
<code>append(s, 20)</code>	4	5	No
<code>append(s, 30)</code>	5	5	No
<code>append(s, 40)</code>	6	10	Yes

`make([]int, 2, 5)` allocates a backing array with capacity 5. The first three `append` calls fit within the existing capacity (len grows $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$). The fourth `append` exceeds capacity 5, so the runtime allocates a new array (typically double, so cap 10) and copies the existing elements. The exact new capacity is implementation-defined but at least 6; in current Go runtimes it is 10.

Exercise 4 (Where is the bug?):

```
package main

import "fmt"

func main() {
    words := []string{"Levitating", "Stay", "Heat Waves", "Stay", "As It Was"}
    var freq map[string]int
    for _, w := range words {
        freq[w]++
    }
    for word, count := range freq {
        if count > 1 {
            fmt.Println(word, count)
        }
    }
}
```

The bug: `var freq map[string]int` declares a nil map. Reading from a nil map returns the zero value (0 for `int`), which is harmless. But **writing to a nil map panics** at runtime. The program panics at `freq[w]++` on the first iteration:

panic: assignment to entry in nil map

Fix: Initialise the map with `make` before the loop:

```
freq := make(map[string]int)
for _, w := range words {
    freq[w]++
}
```

With the fix, the program prints:

Stay 2

Exercise 5 (Write a program): Write a program that reads a slice of song titles and builds a map from the first letter to a slice of titles starting with that letter, then prints each letter and its titles in sorted order.

```
package main

import (
    "fmt"
    "maps"
    "slices"
)

func main() {
    titles := []string{
        "As It Was", "Blinding Lights", "Levitating",
        "Bad Habit", "Kill Bill", "As The World Caves In",
    }
```

```

}

byLetter := make(map[string][]string)
for _, t := range titles {
    letter := string(t[0]) // first byte; safe because all titles start with ASCII
    byLetter[letter] = append(byLetter[letter], t)
}

for _, ts := range byLetter {
    slices.Sort(ts) // sort titles within each group
}

letters := slices.Collect(map.Keys(byLetter))
slices.Sort(letters) // sort the letter keys

for _, letter := range letters {
    fmt.Printf("%s: %v\n", letter, byLetter[letter])
}
}

```

Output:

```

A: [As It Was As The World Caves In]
B: [Bad Habit Blinding Lights]
K: [Kill Bill]
L: [Levitating]

```

Key points: always initialise a map with `make` before writing; `maps.Keys` returns an iterator (Go 1.23+) that `slices.Collect` converts to a sortable slice.

Chapter 8: Interfaces — Answers

Exercise 1 (Think about it): Go’s structural typing means any package can retroactively make its types satisfy an interface defined in any other package. In Java, if you want your `Song` class to satisfy a new interface `Playable` defined in a library you do not control, you must modify `Song`’s source. Explain how Go’s approach changes the relationship between library authors and library users. What does this mean for extending types from packages you cannot modify?

Go’s structural typing means that the author of a type and the author of an interface are completely decoupled. If library A defines `type Track struct { ... }` and later library B defines `interface Playable { Play() }`, and `Track` already has a `Play()` method, then `Track` satisfies `Playable` automatically — neither author needs to know about the other.

For types you cannot modify, the picture is similar: if the type already has the methods you need, you can use it directly where your interface is expected. If it does not, you have two options: wrap the type in your own struct that adds the missing methods (the adapter pattern), or define a new named type based on the original and add methods to that.

This is a fundamental philosophical difference. In Java, the relationship between a class and an interface is declared at write-time and embedded in the source. In Go, the relationship is discovered at compile-time by the compiler — it emerges from what the type does, not from what it says it is. This makes Go code easier to extend and test, because you can define narrow interfaces in your own package that third-party types satisfy without any changes to the third-party source.

Exercise 2 (What does this print?):

```
package main

import "fmt"

type Celsius float64
type Fahrenheit float64

func (c Celsius) String() string {
    return fmt.Sprintf("%.1f°C", float64(c))
}

func printTemp(v fmt.Stringer) {
    fmt.Println(v.String())
}

func main() {
    c := Celsius(37.5)
    f := Fahrenheit(99.5)
    printTemp(c)
    fmt.Println(f)
}
```

Output:

```
37.5°C
99.5
```

`Celsius` has a `String() string` method, so it satisfies `fmt.Stringer`. `printTemp` calls `v.String()` and passes the result to `fmt.Println`, which prints `37.5°C` followed by a newline.

`Fahrenheit` does **not** have a `String() string` method, so it does not satisfy `fmt.Stringer`. `fmt.Println(f)` formats `f` using the default verb for its underlying type, which is `float64`. The default formatting for a float is the shortest decimal representation that rounds back to the same value — here that is `99.5`. No degree symbol, no unit: just the number.

Exercise 3 (Calculation): An interface value in Go stores two fields: a pointer to type information and a pointer to (or copy of) the data. Given a variable declared as `var r io.Reader = &bytes.Buffer{}`, how many distinct type/value components does `r` hold? If `r` is then assigned `nil`, describe the type and value components of the resulting interface value.

`r` holds **two** components: - **Type:** a pointer to the runtime type descriptor for `*bytes.Buffer`. - **Value:** a pointer to the `bytes.Buffer` value on the heap.

After `r = nil`, both components are set to `nil`: - **Type:** `nil` (no concrete type information). - **Value:** `nil` (no data pointer).

This is the **untyped nil** interface value. `r == nil` is `true` after this assignment.

Contrast this with the nil trap in the chapter: if instead you wrote `var buf *bytes.Buffer = nil; r = buf`, the type component would be `*bytes.Buffer` (non-nil) and the value component would be `nil`. That interface value is **not** nil even though `buf` is nil.

Exercise 4 (Where is the bug?):

```

package main

import "fmt"

type DBError struct{ code int }

func (e *DBError) Error() string { return fmt.Sprintf("db error %d", e.code) }

func connect(bad bool) error {
    var err *DBError
    if bad {
        err = &DBError{code: 500}
    }
    return err
}

func main() {
    e := connect(false)
    if e == nil {
        fmt.Println("connected OK")
    } else {
        fmt.Println("failed:", e)
    }
}

```

The bug: connect always returns a non-nil error, even when bad is false.

When bad is false, err is a nil *DBError. The return err statement wraps that typed nil in an error interface value. The interface value has type *DBError (non-nil) and value nil. Because the type component is non-nil, e == nil evaluates to false, and the program prints failed: <nil> instead of connected OK.

The fix:

```

func connect(bad bool) error {
    if bad {
        return &DBError{code: 500}
    }
    return nil // untyped nil: type=nil, value=nil --- this is a true nil error
}

```

Return nil directly rather than returning a typed nil pointer through an interface variable.

Exercise 5 (Write a program):

```

package main

import (
    "fmt"
    "math"
)

type Shape interface {
    Area() float64 // returns the area of the shape
    Perimeter() float64 // returns the perimeter of the shape
}

```



```

type Rectangle struct {
    Width  float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func (r Rectangle) Perimeter() float64 {
    return 2 * (r.Width + r.Height)
}

type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}

func printShapeInfo(s Shape) {
    fmt.Printf("Area:      %.4f\n", s.Area())
    fmt.Printf("Perimeter: %.4f\n", s.Perimeter())
}

func main() {
    r := Rectangle{Width: 4.0, Height: 3.0}
    c := Circle{Radius: 5.0}

    fmt.Println("Rectangle:")
    printShapeInfo(r)

    fmt.Println("Circle:")
    printShapeInfo(c)
}

```

Output:

```

Rectangle:
Area:      12.0000
Perimeter: 14.0000
Circle:
Area:      78.5398
Perimeter: 31.4159

```

Both `Rectangle` and `Circle` satisfy `Shape` implicitly — no declaration required. `printShapeInfo` accepts any `Shape`, so adding a new shape (say, `Triangle`) requires only implementing `Area()` and `Perimeter()` on it; `printShapeInfo` does not change. This is the open/closed principle, Go style.

Chapter 9: Error Handling — Answers

Exercise 1 (Think about it): Java uses checked exceptions to force callers to handle failures. Go returns `error` values that the compiler does not require you to inspect. What are the trade-offs of each approach? In what situations does Go's approach lead to more reliable code, and in what situations might it lead to less reliable code compared to Java's checked exceptions?

Java's checked exceptions make the compiler your partner: if a method declares `throws IOException`, every caller must either catch it or re-declare the throws clause. This guarantees that failure modes are documented in method signatures and that callers cannot silently ignore them — the code will not compile otherwise.

Go takes the opposite position: `error` is a return value like any other. The compiler does not prevent you from discarding it with `_` or simply not capturing it at all. The discipline must come from the programmer and tooling (`errcheck`, `staticcheck`) rather than the language itself.

Where Go tends to win:

- Error handling becomes regular control flow, not exception propagation through a separate, parallel mechanism. Errors can be stored in slices, combined with `errors.Join`, and processed with the same code that handles any other value.
- There is no checked-exception pollution: Java's `throws` clauses ripple upward through call chains, forcing every intermediate method to declare or re-wrap exceptions even when it has nothing meaningful to add. Go functions that merely forward an error just `return err` — no signature change required.
- Errors do not skip stack frames invisibly. The flow of control through a Go program is always traceable by reading the `if err != nil` checks; no hidden stack unwinding occurs.

Where Java's checked exceptions tend to win:

- The compiler catches ignored errors at the call site. A Go developer who writes `n, _ := parseTrackNumber(s)` has silently discarded the error, and the compiler says nothing.
- Checked exceptions create a discoverable, machine-readable contract: the method signature lists every failure mode. Go's error convention requires reading documentation or source code to learn what errors a function may return.
- Refactoring is easier in Java when you add a new failure mode: the compiler identifies every call site that needs updating. In Go, adding a new error condition to a function is invisible to callers.

The bottom line: Go trades compile-time enforcement for simplicity and composability. The approach works well in teams that run linters and review code carefully, and it shines in functions that produce or transform errors as data. It can lead to less reliable code in projects where error checking is informal or tooling is not enforced.

Exercise 2 (What does this print?):

```
package main

import (
    "errors"
    "fmt"
)

var ErrNotFound = errors.New("not found")

type CatalogError struct {
    Track string
    Err   error
}
```

```

func (e *CatalogError) Error() string {
    return fmt.Sprintf("catalog: %s: %s", e.Track, e.Err)
}

func (e *CatalogError) Unwrap() error {
    return e.Err
}

func lookup(track string) error {
    return &CatalogError{Track: track, Err: ErrNotFound}
}

func main() {
    err := lookup("Tití Me Preguntó")
    fmt.Println(err)
    fmt.Println(errors.Is(err, ErrNotFound))

    var ce *CatalogError
    if errors.As(err, &ce) {
        fmt.Println(ce.Track)
    }
}

```

Output:

```

catalog: Tití Me Preguntó: not found
true
Tití Me Preguntó

```

lookup("Tití Me Preguntó") returns a *CatalogError with Track = "Tití Me Preguntó" and Err = ErrNotFound.

fmt.Println(err) calls err.Error(), which returns "catalog: Tití Me Preguntó: not found". fmt.Println appends a newline, so the first line of output is catalog: Tití Me Preguntó: not found.

errors.Is(err, ErrNotFound) starts at err (a *CatalogError) and calls == against ErrNotFound — no match. It then calls err.Unwrap(), which returns ErrNotFound itself. ErrNotFound == ErrNotFound is true. So errors.Is returns true, and fmt.Println(true) prints true.

errors.As(err, &ce) checks whether err is assignable to *CatalogError. It is, so ce is set to the *CatalogError value and errors.As returns true. The if body prints ce.Track, which is "Tití Me Preguntó".

Exercise 3 (Calculation): Consider the following code. How many distinct, non-nil error values does validateSong return for the input Song{Title: "", Artist: "Karol G", Year: 2021, BPM: -1}? What is the output of fmt.Println(err) for that input?

```

package main

import (
    "errors"
    "fmt"
)

type Song struct {
    Title string

```

```

    Artist string
    Year    int
    BPM     int
}

func validateSong(s Song) error {
    var errs []error
    if s.Title == "" {
        errs = append(errs, errors.New("title required"))
    }
    if s.Year < 2000 || s.Year > 2030 {
        errs = append(errs, fmt.Errorf("year %d out of range", s.Year))
    }
    if s.BPM <= 0 {
        errs = append(errs, errors.New("BPM must be positive"))
    }
    return errors.Join(errs...)
}

func main() {
    s := Song{Title: "", Artist: "Karol G", Year: 2021, BPM: -1}
    err := validateSong(s)
    fmt.Println(err)
}

```

Answer: `validateSong` collects 2 distinct, non-nil error values.

Trace through the conditions for `Song{Title: "", Artist: "Karol G", Year: 2021, BPM: -1}`:

- `s.Title == ""` is true — `errors.New("title required")` is appended. (1 error)
- `s.Year < 2000 || s.Year > 2030`: `2021 < 2000` is false; `2021 > 2030` is false — condition is false, no error appended.
- `s.BPM <= 0`: `-1 <= 0` is true — `errors.New("BPM must be positive")` is appended. (2 errors)

`errors.Join` receives a slice of 2 non-nil errors. Its `Error()` method joins their messages with a newline between them.

Output:

```

title required
BPM must be positive

```

Note that `Artist` has no validation rule, so "Karol G" (a valid, non-empty value) does not contribute any error. `Year = 2021` falls within the range `[2000, 2030]`, so no year error is produced either.

Exercise 4 (Where is the bug?):

```

package main

import (
    "errors"
    "fmt"
    "io"
)

func readAll(r io.Reader) ([]byte, error) {
    buf := make([]byte, 4)

```

```

var result []byte
for {
    n, err := r.Read(buf)
    result = append(result, buf[:n]...)
    if err == io.EOF {
        break
    }
    if err != nil {
        return nil, fmt.Errorf("readAll: %w", err)
    }
}
return result, nil
}

func main() {
    r := strings.NewReader("TQG")
    data, err := readAll(r)
    if err != nil {
        fmt.Println("error:", err)
        return
    }
    fmt.Println(string(data))
}

```

The bug: There are actually two problems.

Bug 1 — missing import: `strings.NewReader` is used in `main` but `"strings"` is not in the import list. The program will not compile. The import block should be:

```

import (
    "errors"
    "fmt"
    "io"
    "strings"
)

```

Bug 2 — comparing `err == io.EOF` directly instead of using `errors.Is`: The sentinel check `if err == io.EOF` works correctly for `io.EOF` itself, but it will silently miss `io.EOF` if the reader ever wraps it (e.g., `fmt.Errorf("read: %w", io.EOF)`). The idiomatic fix is:

```

if errors.Is(err, io.EOF) {
    break
}

```

Using `errors.Is` is consistent, future-proof, and is what the chapter recommends. The `errors` import is already present, so this is a zero-cost change.

Corrected `readAll`:

```

func readAll(r io.Reader) ([]byte, error) {
    buf := make([]byte, 4)
    var result []byte
    for {
        n, err := r.Read(buf)
        result = append(result, buf[:n]...)
        if errors.Is(err, io.EOF) {
            break
        }
    }
}

```

```

        if err != nil {
            return nil, fmt.Errorf("readAll: %w", err)
        }
    }
    return result, nil
}

```

With both fixes applied, the program compiles and prints:

TQG

Exercise 5 (Write a program):

```
package main
```

```
import (
    "errors"
    "fmt"
    "strconv"
    "strings"
)
```

```
var ErrInvalidTimecode = errors.New("invalid timecode")
```

```
func parseTimecode(s string) (int, int, int, error) {
    parts := strings.Split(s, ":")
    if len(parts) != 2 {
        return 0, 0, 0, fmt.Errorf("parseTimecode: expected MM:SS, got %q: %w", s, ErrInvalidTimecode)
    }

    minutes, err := strconv.Atoi(parts[0])
    if err != nil {
        return 0, 0, 0, fmt.Errorf("parseTimecode: invalid minutes %q: %w", parts[0], ErrInvalidTimecode)
    }

    seconds, err := strconv.Atoi(parts[1])
    if err != nil {
        return 0, 0, 0, fmt.Errorf("parseTimecode: invalid seconds %q: %w", parts[1], ErrInvalidTimecode)
    }

    if minutes < 0 {
        return 0, 0, 0, fmt.Errorf("parseTimecode: minutes %d is negative: %w", minutes, ErrInvalidTimecode)
    }
    if seconds < 0 || seconds > 59 {
        return 0, 0, 0, fmt.Errorf("parseTimecode: seconds %d out of range [0,59]: %w", seconds, ErrInvalidTimecode)
    }

    total := minutes*60 + seconds
    return minutes, seconds, total, nil
}

```

```
func main() {
    inputs := []string{"03:45", "345", "01:61"}
}

```

```

for _, tc := range inputs {
    m, s, total, err := parseTimecode(tc)
    if err != nil {
        fmt.Printf("%-10s => error: %s\n", tc, err)
        fmt.Printf("                is ErrInvalidTimecode: %v\n", errors.Is(err, ErrInvalidTimecode))
    } else {
        fmt.Printf("%-10s => %dm %ds (%d total seconds)\n", tc, m, s, total)
    }
}
}

```

Output:

```

03:45      => 3m 45s (225 total seconds)
345        => error: parseTimecode: expected MM:SS, got "345": invalid timecode
           is ErrInvalidTimecode: true
01:61      => error: parseTimecode: seconds 61 out of range [0,59]: invalid timecode
           is ErrInvalidTimecode: true

```

Key design decisions explained:

- `ErrInvalidTimecode` is a package-level sentinel declared with `errors.New`. Exporting it (capital E) lets callers in other packages use `errors.Is` to distinguish timecode errors from other error kinds.
- Every error path uses `fmt.Errorf("...: %w", ErrInvalidTimecode)` to wrap the sentinel. This means the returned error has a human-readable message that includes the context (the bad input, the specific reason) **and** a chain that `errors.Is` can walk to find `ErrInvalidTimecode`.
- The function returns four values: (`int`, `int`, `int`, `error`). The three `int` values are zero on error, consistent with the Go convention of returning zero values alongside a non-nil error.
- `strings.Split(s, ":")` with a check on `len(parts) != 2` is the idiomatic way to parse a two-part format. Using `fmt.Sscanf` or a regex are also valid; `strings.Split` is the most readable for this simple case.

Chapter 10: Goroutines and Channels — Answers

Exercise 1 (Think about it): Java’s `Thread` and `Runnable` model requires you to think about thread pool sizing. Go’s goroutine model mostly frees you from this. Explain the runtime mechanism that makes goroutines cheap enough to use one per task. What cost, if any, do goroutines impose that Java threads do not, and when might you still want to limit the number of running goroutines?

The key mechanism is **M:N scheduling**: the Go runtime multiplexes M goroutines onto N OS threads, where N defaults to the number of CPU cores (`GOMAXPROCS`). The scheduler lives in user space, so switching between goroutines does not require a kernel mode transition — it is many times faster than a Java thread context switch.

Goroutines start with a ~2 KB stack that grows dynamically as needed (up to a configurable maximum, typically 1 GB). Java threads allocate their full stack (512 KB to 1 MB) up front, from virtual memory at minimum. This means creating a million goroutines consumes roughly 2 GB of initial stack memory; creating a million Java threads would require 500 GB to 1 TB. In practice, the OS would refuse long before that.

Costs goroutines impose: Each goroutine is a heap allocation tracked by the scheduler. At very high goroutine counts (hundreds of thousands) the scheduler itself becomes a bottleneck, and GC pressure increases because goroutine stacks are heap-allocated. There is also some overhead per goroutine in the runtime’s internal bookkeeping structures.

When to still limit goroutines: Any time the goroutines are doing I/O-bound work that creates downstream resource pressure — for example, goroutines that each open a database connection or a file descriptor. Even if the goroutines themselves are cheap, the external resources they consume (connections, file descriptors, memory for outbound HTTP requests) are not. The common Go idiom for bounding concurrency is a buffered channel used as a semaphore, or the worker-pool pattern covered in Chapter 13.

Exercise 2 (What does this print?):

```
package main

import "fmt"

func main() {
    ch := make(chan int, 3)

    ch <- 7
    ch <- 13
    ch <- 21
    close(ch)

    for v := range ch {
        fmt.Println(v)
    }

    v, ok := <-ch
    fmt.Println(v, ok)
}
```

Output:

```
7
13
21
0 false
```

Why:

The channel has capacity 3, so all three sends succeed without blocking — no goroutine is needed. `close(ch)` marks the channel closed; the three buffered values are still available to receive.

`range ch` drains the channel in FIFO order, printing 7, 13, and 21. When the buffer is empty and the channel is closed, `range` terminates.

After the loop, `<-ch` receives from a channel that is both closed and empty. The comma-ok idiom returns the **zero value** of the element type (0 for `int`) and `false` for `ok`, because the channel is exhausted. `fmt.Println(v, ok)` prints `0 false`.

This demonstrates two important rules: buffered values survive a `close`, and receiving from an empty closed channel always returns (**zero, false**) rather than blocking or panicking.

Exercise 3 (Calculation): Consider the following program. Trace its execution and determine the exact output. How many goroutines are alive (other than `main`) when the final `fmt.Println` in `main` runs?

```
package main

import "fmt"
```



```

func double(in <-chan int, out chan<- int) {
    for v := range in {
        out <- v * 2
    }
    close(out)
}

func main() {
    src := make(chan int, 3)
    dst := make(chan int, 3)

    src <- 3
    src <- 5
    src <- 8
    close(src)

    go double(src, dst)

    for result := range dst {
        fmt.Println(result)
    }
    fmt.Println("done")
}

```

Output:

```

6
10
16
done

```

Trace:

1. `src` is a buffered channel with capacity 3. The three sends (3, 5, 8) all succeed immediately without blocking. `src` is then closed.
2. `go double(src, dst)` launches `double` as a goroutine. `double` reads from `src` using `range`, which drains the buffered values 3, 5, 8 in order and then exits when `src` is empty and closed. For each value, it sends the doubled result to `dst` (also buffered with capacity 3, so no blocking occurs). After the loop, `double` calls `close(dst)`.
3. Back in `main`, for `result := range dst` drains `dst`. The values arrive in order: 6, 10, 16. When `dst` is closed and empty, the loop ends.
4. `fmt.Println("done")` runs last.

Goroutines alive when `fmt.Println("done")` runs: Zero (other than `main`). The `double` goroutine has already returned — it finished draining `src`, called `close(dst)`, and exited before `main`'s `range dst` loop could finish (since `close(dst)` is what caused the loop to terminate).

Exercise 4 (Where is the bug?):

```

package main

import (
    "fmt"

```

```

    "sync"
)

func main() {
    var wg sync.WaitGroup
    results := make(chan string, 3)
    tracks := []string{"rockstar", "Circles", "Earfquake"}

    for _, t := range tracks {
        wg.Add(1)
        go func() {
            defer wg.Done()
            results <- "Playing: " + t
        }()
    }

    wg.Wait()
    close(results)

    for r := range results {
        fmt.Println(r)
    }
}

```

The bug: The goroutine closure captures the loop variable `t` by reference, not by value. By the time the goroutines run, the `for` loop has advanced `t` to the last value in the slice. All three goroutines read the same final value — "Earfquake" — and send it three times. The output is likely:

```

Playing: Earfquake
Playing: Earfquake
Playing: Earfquake

```

This is the **loop-closure capture bug** described in Chapter 5.

Why it happens: In Go, the range variable `t` is a single variable whose value is updated on each iteration. All three goroutines close over the same `t` variable (a single memory address), not over a copy of its value at the time the goroutine was launched. Because the goroutines are scheduled after the loop completes, `t` holds the last assigned value when they execute.

The fix: Capture the value at goroutine launch time by passing it as a parameter to the anonymous function, or by introducing a local copy:

```

for _, t := range tracks {
    t := t // new variable scoped to this iteration
    wg.Add(1)
    go func() {
        defer wg.Done()
        results <- "Playing: " + t
    }()
}

```

Or equivalently, pass `t` as a function argument:

```

for _, t := range tracks {
    wg.Add(1)
    go func(track string) {
        defer wg.Done()
        results <- "Playing: " + track
    }(t)
}

```

```
    }(t)
}
```

Both fixes capture the value of `t` at the point of goroutine creation so each goroutine gets its own independent copy.

Note: In Go 1.22 and later, range variables are per-iteration by default, which eliminates this class of bug automatically. If you are on Go 1.22 or newer, the original code would work correctly. On earlier versions, the fix is required.

Exercise 5 (Write a program):

```
package main

import (
    "fmt"
    "time"
)

func main() {
    ch1 := make(chan string, 1)
    ch2 := make(chan string, 1)
    ch3 := make(chan string, 1)

    go func() {
        time.Sleep(10 * time.Millisecond)
        ch1 <- "Post Malone: Circles"
    }()
    go func() {
        time.Sleep(20 * time.Millisecond)
        ch2 <- "Tyler: Wilder World"
    }()
    go func() {
        time.Sleep(30 * time.Millisecond)
        ch3 <- "Post Malone: rockstar"
    }()

    received := 0
    total := 3
    for received < total {
        select {
            case msg := <-ch1:
                fmt.Println(msg)
                received++
            case msg := <-ch2:
                fmt.Println(msg)
                received++
            case msg := <-ch3:
                fmt.Println(msg)
                received++
            case <-time.After(100 * time.Millisecond):
                fmt.Println("timeout")
                received = total // exit the loop
        }
    }
}
```

```
}
}
```

Output (order reflects goroutine sleep durations):

```
Post Malone: Circles
Tyler: Wilder World
Post Malone: rockstar
```

How it works:

Each goroutine sleeps for a different duration before sending on its dedicated channel. `main` loops using `select`, blocking until any of the four cases is ready. Because the goroutines sleep for 10 ms, 20 ms, and 30 ms, the messages arrive in that order.

The `time.After(100 * time.Millisecond)` case provides a safety net. `time.After` returns a receive-only channel (`<-chan time.Time`) that the `time` package sends a value on after the specified duration. If no message arrives within 100 ms, that case fires, prints "timeout", and sets `received = total` to exit the loop.

A subtle point: `time.After` creates a new timer on every call to `select`, which is fine for correctness but slightly wasteful. In production code that needs tight control over timer lifetimes, you would create a `time.NewTimer` once and reuse it. That is a concern for Chapter 15; the `time.After` form is idiomatic for simple timeouts.

Chapter 11: Synchronization — Answers

Exercise 1 (Think about it): Java's `synchronized` keyword locks an object's monitor, which is built into every Java object. Go has no per-object monitor; instead you declare explicit `sync.Mutex` fields. What are the practical advantages and disadvantages of each approach? Consider: what happens when you need to protect two independent fields in the same struct, and how would you do it with each language's mechanism?

Java's per-object monitor is convenient for simple cases: every object already has a lock, so you can write `synchronized (this)` with no extra declarations. The downside is that the monitor is coarse-grained — there is only one per object. If a struct (class in Java) has two independent fields that can be updated concurrently without affecting each other, locking the whole object monitor for either field creates unnecessary contention. Java programmers work around this with separate `java.util.concurrent.locks.Lock` objects or by using a dedicated inner lock object:

```
private final Object tracksLock = new Object();
private final Object playsLock  = new Object();

synchronized (tracksLock) { tracks.add(track); }
synchronized (playsLock)  { plays.increment(); }
```

Go's approach makes this natural: you simply declare two independent mutex fields.

```
type Catalog struct {
    tracksMu sync.Mutex
    tracks   []string

    playsMu sync.Mutex
    plays   map[string]int
}
```

Each mutex protects only the field it is paired with, and neither blocks the other. This is less magic but more explicit.

The practical advantages of Go's approach:

- **Granularity:** You can have as many independent mutexes as you need at zero structural cost.
- **Clarity:** The pairing between a mutex and the data it protects is visible in the struct definition.
- **No accidental sharing:** In Java, every synchronized method on the same object uses the same monitor, even if they protect unrelated state. In Go, each mutex is independent by default.

The practical disadvantage:

- **Verbosity:** You must declare, name, and document each mutex. Java's implicit monitor requires no declaration.
- **Copy hazard:** Go structs are value types. Copying a struct that contains a `sync.Mutex` is a bug; the struct must always be passed and stored by pointer. Java objects are always references, so this hazard does not exist.

Exercise 2 (What does this print?):

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var once sync.Once
    var wg sync.WaitGroup
    results := make([]string, 3)

    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            once.Do(func() {
                results[n] = "loaded"
            })
            if results[n] == "" {
                results[n] = "skipped"
            }
        }(i)
    }

    wg.Wait()
    loaded := 0
    skipped := 0
    for _, r := range results {
        if r == "loaded" {
            loaded++
        } else if r == "skipped" {
            skipped++
        }
    }
    fmt.Printf("loaded=%d skipped=%d\n", loaded, skipped)
}
```

The output is:

```
loaded=1 skipped=2
```

Here is why.

`sync.Once` guarantees that the function passed to `Do` runs **exactly once** across all goroutines. One of the three goroutines (say goroutine with `n=0`, `n=1`, or `n=2` — the scheduler decides which wins) will execute `results[n] = "loaded"`, setting one slot of the `results` slice.

The other two goroutines call `once.Do` as well, but their function bodies are silently dropped because the `once` is already done. They proceed past `once.Do` and check `results[n] == ""` for their own slot `n`. Because the winning goroutine wrote to a **different** index than these two, their slots are still empty, so they set `results[n] = "skipped"`.

The final tally is always exactly one "loaded" and two "skipped", regardless of which goroutine wins the `once.Do` race.

Note: even though goroutines access different indices of `results` concurrently, this specific program is **not** a data race because each goroutine always writes to its own `results[n]` (where `n` is passed by value), and no two goroutines write to the same index.

Exercise 3 (Calculation):

```
var counter atomic.Int64
var wg sync.WaitGroup

for i := 0; i < 4; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        counter.Add(10)
    }()
}
wg.Wait()
fmt.Println(counter.Load())
```

(a) `counter.Load()` always prints 40.

`atomic.Int64.Add` is an atomic read-modify-write operation. No matter what order the four goroutines execute, each `Add(10)` is applied to the current value atomically, and all four additions will complete before `wg.Wait()` returns. The final value is always $4 \times 10 = 40$. This would **not** be true with a plain `int` counter and no synchronization — that would be a data race with unpredictable results.

(b) Replacing `counter.Add(10)` with `counter.Add(int64(i))` introduces a **closure capture bug**.

The goroutine closure captures the **variable** `i`, not its value at the moment the goroutine was launched. By the time the goroutines run, the loop may have already incremented `i` past the value it had when `go func()` was called. In the worst case, all four goroutines see `i == 4` (the value after the loop ends) and print $4 * 4 = 16$. In the best case, they each capture a different value (0, 1, 2, 3) and print $0 + 1 + 2 + 3 = 6$. Any value between 0 and 16 is possible, and the result is non-deterministic.

The fix is the same as described in Chapter 5 (closures): pass `i` as a parameter to the goroutine function.

```
go func(n int) {
    defer wg.Done()
    counter.Add(int64(n))
}(i) // pass i by value here
```

With this fix the result is always $0 + 1 + 2 + 3 = 6$.

Exercise 4 (Where is the bug?):

```
type SafeMap struct {
    mu sync.Mutex
    m  map[string]int
}

func NewSafeMap() SafeMap {
    return SafeMap{m: make(map[string]int)}
}

func (s SafeMap) Inc(key string) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.m[key]++
}

func (s SafeMap) Get(key string) int {
    s.mu.Lock()
    defer s.mu.Unlock()
    return s.m[key]
}
```

The bug is that `Inc` and `Get` have **value receivers** (`s SafeMap`), not pointer receivers (`s *SafeMap`).

When a method has a value receiver, Go passes a **copy** of the struct. Each call to `Inc` locks the mutex in its own private copy — a different mutex instance than the one in `sm` in `main`. The lock is acquired and released on a throwaway copy, providing no mutual exclusion on the real `sm`. One hundred goroutines therefore write to `sm.m["Butter"]` concurrently without any synchronization, which is a data race.

The map itself (`sm.m`) is a reference type, so the map operations do land on the shared map — but they are completely unprotected, and concurrent writes to a Go map without synchronization is undefined behavior (the runtime will panic with a “concurrent map writes” message).

The fix is to use pointer receivers throughout:

```
func (s *SafeMap) Inc(key string) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.m[key]++
}

func (s *SafeMap) Get(key string) int {
    s.mu.Lock()
    defer s.mu.Unlock()
    return s.m[key]
}
```

And in `main`, take the address of `sm` (or change `NewSafeMap` to return `*SafeMap`):

```
func NewSafeMap() *SafeMap {
    return &SafeMap{m: make(map[string]int)}
}

func main() {
    sm := NewSafeMap() // *SafeMap; no copy needed
```

```

    ...
    sm.Inc("Butter")
    ...
    fmt.Println(sm.Get("Butter")) // 100
}

```

With pointer receivers and a pointer variable, every call to `Inc` and `Get` locks the **same** mutex, and the output is reliably 100.

Exercise 5 (Write a program): Implement a concurrent-safe `RateLimiter` struct that uses a `sync.Mutex` to protect a counter and a `time.Time` field tracking when the window resets. The struct should have a method `Allow(n int) bool` that returns `true` if `n` tokens are available in the current one-second window, deducting them if so, and `false` otherwise (without deducting). Write a `main` function that launches 10 goroutines, each calling `Allow(1)` in a loop 5 times, and prints how many calls were allowed versus denied across all goroutines combined. Use `sync.WaitGroup` to wait for all goroutines to finish.

```

package main

import (
    "fmt"
    "sync"
    "sync/atomic"
    "time"
)

// RateLimiter allows at most Limit tokens per one-second window.
type RateLimiter struct {
    mu      sync.Mutex
    limit   int
    used    int
    resetAt time.Time
}

func NewRateLimiter(limit int) *RateLimiter {
    return &RateLimiter{
        limit:   limit,
        resetAt: time.Now().Add(time.Second),
    }
}

// Allow returns true and deducts n tokens if they are available.
// It returns false without deducting if the window is exhausted.
func (r *RateLimiter) Allow(n int) bool {
    r.mu.Lock()
    defer r.mu.Unlock()

    now := time.Now()
    if now.After(r.resetAt) {
        r.used = 0
        r.resetAt = now.Add(time.Second)
    }

    if r.used+n > r.limit {
        return false
    }
}

```



```

    }
    r.used += n
    return true
}

func main() {
    limiter := NewRateLimiter(25) // allow 25 calls per second
    var wg sync.WaitGroup
    var allowed, denied atomic.Int64

    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for j := 0; j < 5; j++ {
                if limiter.Allow(1) {
                    allowed.Add(1)
                } else {
                    denied.Add(1)
                }
            }
        }()
    }

    wg.Wait()
    fmt.Printf("allowed=%d denied=%d total=%d\n",
        allowed.Load(), denied.Load(), allowed.Load()+denied.Load())
}

```

Sample output (with a limit of 25 and 50 total calls):

```
allowed=25 denied=25 total=50
```

Key points of the implementation:

- The `sync.Mutex` in `RateLimiter` protects both `used` and `resetAt` together as a single invariant. Neither field can be read or written outside the lock.
- `Allow` checks the current time inside the lock so that the window reset and the token deduction are one atomic decision. If the check and the deduction were in separate lock acquisitions, another goroutine could sneak in between them.
- The `allowed` and `denied` counters in `main` use `atomic.Int64` rather than a mutex because they are independent single-variable updates — a perfect atomic use case.
- `wg.Add(1)` is called in the outer loop, before the goroutine is launched, not inside the goroutine — following the `WaitGroup` rule from the chapter.

Chapter 12: Context and Concurrency Patterns — Answers

Exercise 1 (Think about it): In Java, cancelling an in-flight operation typically means calling `Future.cancel(true)` or interrupting a thread via `Thread.interrupt()`. Describe how Go's `context.Context` model differs from Java's thread-interrupt approach. What are the advantages of passing a context explicitly rather than relying on a thread-level interrupt mechanism? Consider what happens when a Java thread is blocked in a third-party library that does not handle `InterruptedException`, compared to how a Go function using a context-aware library would behave.

Java's thread-interrupt model is **implicit and cooperative at the thread level**. When you call `Thread.interrupt()`, a flag is set on the thread, and blocking calls like `Object.wait()`, `Thread.sleep()`, and `java.io.InputStream.read()` on some implementations throw `InterruptedException` when they notice it. But not every blocking operation checks the flag: a thread blocked in a native call, a third-party lock, or a legacy `InputStream` implementation may never see the interrupt at all. The interrupt propagates up the call stack only as long as every layer catches and re-throws (or re-sets) the flag, which is notoriously easy to accidentally swallow:

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // oops, swallowed it; the interrupt flag is now cleared
}
```

Go's `context.Context` is **explicit and uniform**. Every function that can be cancelled must accept a `context.Context` parameter. Cancellation is communicated by closing `ctx.Done()`, which is observable without any thread-local state. Any function that calls another context-aware function simply passes the same context through; the propagation is visible in every function signature.

The advantages over thread interrupts are:

1. **Explicit propagation.** You can see in the function signature that a function is cancellable. In Java, there is no signature-level signal that a method checks `Thread.interrupted()`.
2. **Deadlines and timeouts as first-class values.** `context.WithTimeout` and `context.WithDeadline` associate a deadline with the context object itself, not with a thread. Multiple goroutines can share the same context and respect the same deadline without any shared mutable state.
3. **No accidental swallowing.** Because `ctx.Done()` is a channel, you either select on it or you do not — there is no exception to catch and accidentally discard.
4. **Composability.** Derived contexts (`WithCancel`, `WithTimeout`) form a tree. Cancelling a parent automatically cancels all children. Java's thread-interrupt model is flat: each thread has exactly one interrupt flag.
5. **Request-scoped values.** `context.WithValue` lets you attach metadata (trace IDs, auth tokens) to a context and retrieve it anywhere in the call tree without global state.

If a Java thread is blocked in a third-party library that does not handle `InterruptedException` — for example, a legacy JDBC driver — calling `Thread.interrupt()` may have no effect. The thread stays blocked, and the only recourse is to close the underlying socket from another thread or wait for the operation to time out at the OS level. A Go function calling a database driver built on top of `database/sql` passes a context to `db.QueryContext`; the driver layer itself monitors `ctx.Done()` and closes the connection if the context is cancelled. The library author opts in once; all callers benefit automatically.

Exercise 2 (What does this print?):

```
package main

import (
    "context"
    "fmt"
    "time"
)

func work(ctx context.Context, label string) {
    select {
    case <-time.After(500 * time.Millisecond):
        fmt.Println(label, "done")
    case <-ctx.Done():
```

```

        fmt.Println(label, "cancelled:", ctx.Err())
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
    defer cancel()

    go work(ctx, "Bewitched")
    go work(ctx, "Too Sweet")
    time.Sleep(400 * time.Millisecond)
    fmt.Println("main done")
}

```

Output (order of the first two lines may vary):

```

Bewitched cancelled: context deadline exceeded
Too Sweet cancelled: context deadline exceeded
main done

```

The context has a 200 ms timeout. Both `work` goroutines are launched immediately and block in their `select` statement waiting for either `time.After(500ms)` or `ctx.Done()`. After 200 ms the timeout fires, `ctx.Done()` is closed, and both goroutines unblock on the `ctx.Done()` case. Each prints its label with "cancelled: context deadline exceeded". The goroutines finish well before `main`'s `time.Sleep(400ms)` elapses, so "main done" appears last.

The two cancelled lines may appear in either order because goroutine scheduling is not deterministic. `main done` always appears last because `time.Sleep(400ms)` is longer than the 200 ms timeout and the goroutines' response time.

Exercise 3 (Calculation): You run a worker pool with `workers = 3` and feed it a slice of 7 tasks. Each task takes exactly 100 ms. Assuming no overhead and perfect parallelism, how many milliseconds does the pool take to complete all 7 tasks?

Answer: 300 ms.

With 3 workers processing tasks that each take 100 ms:

Round	Tasks processed	Wall-clock time elapsed
1	tasks 1, 2, 3	0 – 100 ms
2	tasks 4, 5, 6	100 – 200 ms
3	task 7 (+ 2 idle)	200 – 300 ms

Round 1 dispatches tasks 1–3 in parallel. All three finish at $T=100$ ms. Round 2 dispatches tasks 4–6 in parallel; all finish at $T=200$ ms. Round 3 dispatches task 7 alone (only one task remains); it finishes at $T=300$ ms.

Total elapsed time = $\text{ceil}(7 / 3) \times 100 \text{ ms} = 3 \times 100 \text{ ms} = \mathbf{300 \text{ ms}}$.

General formula: `ceil(N / workers) * task_duration`.

Exercise 4 (Where is the bug?):

```
package main
```

```

import (
    "context"
    "fmt"
    "time"
)

func fetchData(url string) <-chan string {
    ch := make(chan string)
    go func() {
        time.Sleep(2 * time.Second)
        ch <- "result for " + url
    }()
    return ch
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 500*time.Millisecond)
    defer cancel()

    ch := fetchData("https://example.com/songs")
    select {
    case result := <-ch:
        fmt.Println(result)
    case <-ctx.Done():
        fmt.Println("timed out")
    }
}

```

The bug: goroutine leak in fetchData.

fetchData launches a goroutine that sleeps for 2 seconds and then sends on `ch`. When the context times out after 500 ms, `main` exits the `select` via `ctx.Done()` and prints "timed out". At this point `ch` is no longer being read by anyone. The goroutine inside `fetchData` is still sleeping; when it wakes up at $T=2$ s and tries to send `ch <- "result for ..."`, it blocks forever because nobody will ever receive from `ch`. The goroutine is leaked — it will never exit.

The fix: pass the context into `fetchData` so the goroutine can bail out early.

```

func fetchData(ctx context.Context, url string) <-chan string {
    ch := make(chan string, 1) // buffered so the goroutine can send even if nobody reads
    go func() {
        select {
        case <-time.After(2 * time.Second):
            ch <- "result for " + url // send result if we finish in time
        case <-ctx.Done():
            // context was cancelled; exit cleanly without sending
        }
    }()
    return ch
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 500*time.Millisecond)
    defer cancel()

    ch := fetchData(ctx, "https://example.com/songs")
}

```

```

    select {
    case result := <-ch:
        fmt.Println(result)
    case <-ctx.Done():
        fmt.Println("timed out")
    }
}

```

Using a buffered channel of capacity 1 also guards against a secondary leak: if the result arrives after `main`'s `select` exits the `ctx.Done()` branch (a narrow race), the goroutine can still send on `ch` without blocking, and then exit.

Exercise 5 (Write a program):

```

package main

import (
    "context"
    "fmt"
    "math/rand"
    "time"

    "golang.org/x/sync/errgroup"
)

// fanOutFetch fetches all song titles concurrently using errgroup.
// Each fetch is simulated with a random sleep between 50 and 150 ms.
// The function returns the titles in the same order as songs, or an error
// if the context is cancelled before all fetches complete.
func fanOutFetch(ctx context.Context, songs []string) ([]string, error) {
    results := make([]string, len(songs))
    g, ctx := errgroup.WithContext(ctx)

    for i, song := range songs {
        i, song := i, song // capture for Go < 1.22
        g.Go(func() error {
            delay := time.Duration(50+rand.Intn(100)) * time.Millisecond
            select {
            case <-time.After(delay):
                results[i] = "fetched: " + song
                return nil
            case <-ctx.Done():
                return ctx.Err()
            }
        })
    }

    if err := g.Wait(); err != nil {
        return nil, err
    }
    return results, nil
}

func main() {

```

```

songs := []string{
    "From The Start",
    "Bewitched",
    "Too Sweet",
    "Work Song",
}

ctx, cancel := context.WithTimeout(context.Background(), 300*time.Millisecond)
defer cancel()

results, err := fanOutFetch(ctx, songs)
if err != nil {
    fmt.Println("error:", err)
    return
}
for _, r := range results {
    fmt.Println(r)
}
}

```

Explanation:

`errgroup.WithContext` derives a new context from the one passed in. If any goroutine returns a non-nil error, `errgroup` cancels that derived context, causing all other goroutines that are still sleeping to unblock on `ctx.Done()` and return `ctx.Err()`. `g.Wait()` returns the first error.

Because the outer `context.WithTimeout` fires after 300 ms, any fetch whose random delay exceeds the remaining budget will be cancelled. Fetches with delays in the 50–150 ms range should all complete well within 300 ms under normal conditions; set the timeout lower (e.g., 100 ms) to reliably trigger a cancellation in testing.

Sample output when all fetches succeed:

```

fetched: From The Start
fetched: Bewitched
fetched: Too Sweet
fetched: Work Song

```

Sample output when the timeout fires:

```

error: context deadline exceeded

```

Chapter 13: Packages and Modules — Answers

Exercise 1 (Think about it): Maven and Gradle resolve transitive dependencies automatically and let two artifacts declare conflicting version requirements for the same library. They use a strategy (nearest-wins in Maven, highest-requested in Gradle) to pick a single version at build time. Go’s module system takes a different approach called Minimum Version Selection (MVS): it always picks the minimum version that satisfies all requirements. Compare these two philosophies. What problems does MVS avoid? What does it make harder? When might the Go approach cause a surprise after running `go get pkg@latest`?

Go’s Minimum Version Selection works by computing the maximum of the minimum required versions across all modules in the dependency graph. If module A requires `library v1.2.0` and module B requires `library v1.3.0`, Go selects `v1.3.0` — the minimum version that satisfies both. No module ever gets a version newer than the one its author tested against, unless someone explicitly requests an upgrade.

Problems MVS avoids:

- **Silent upgrades.** In Maven's nearest-wins model, adding a new dependency can silently pull in a newer (or older) version of a transitive library, breaking unrelated code. MVS never introduces a version you did not ask for.
- **Build irreproducibility.** Because MVS is deterministic and recorded in `go.sum`, two developers checking out the same commit always get bit-for-bit identical dependencies. Maven can produce different builds depending on which dependencies happen to be in the local repository cache.

What MVS makes harder:

- **Staying current.** MVS actively resists upgrading. If your dependency graph has pinned a library at `v1.2.0`, you will stay there until someone runs `go get library@v1.4.0`. In a large organisation this can mean security patches go unnoticed.
- **Downgrading.** If you want to use an older version than the graph currently requires, you have to remove or downgrade every module that requires the newer version.

Surprise from `go get pkg@latest`: After you run `go get pkg@latest`, the upgraded module may itself require newer versions of transitive dependencies. MVS will bump those transitives to the versions the new module requires — which might be substantially newer than before. Your `go.mod` can change in unexpected ways beyond the single module you asked to upgrade. Running `go mod tidy` afterward and reviewing the diff in `go.mod` and `go.sum` is a good habit.

Exercise 2 (What does this print?):

Given the following three files in a module `github.com/zachbryan/demo`:

File `lyrics/lyrics.go`:

```
package lyrics

import "fmt"

func Print() {
    fmt.Println("something in the orange")
}
```

File `lyrics/internal/detail/detail.go`:

```
package detail

import "fmt"

func Show() {
    fmt.Println("internal detail")
}
```

File `main.go`:

```
package main

import (
    "github.com/zachbryan/demo/lyrics"
    "github.com/zachbryan/demo/lyrics/internal/detail"
)

func main() {
    lyrics.Print()
}
```

```
    detail.Show()
}
```

What happens when you run `go build`? If the build succeeds, what does the program print? If not, explain why.

The build fails.

`main.go` is at the module root, which means its parent directory for the purposes of the `internal` rule is `github.com/zachbryan/demo`. The `internal` package's full path is `github.com/zachbryan/demo/lyrics/internal/detail`. For `main.go` to import it, `main.go` must live inside `github.com/zachbryan/demo/lyrics` or one of its subdirectories. `main.go` lives at the module root, which is `github.com/zachbryan/demo` — it is not rooted under `github.com/zachbryan/demo/lyrics`, so the compiler rejects the import.

The compiler error will say something like:

```
use of internal package github.com/zachbryan/demo/lyrics/internal/detail not allowed
```

The import of `github.com/zachbryan/demo/lyrics` (the public package) is fine. Only the `internal/detail` import is rejected.

To fix this, either move `detail` out of `lyrics/internal/` into a location that `main.go` is allowed to reach (such as `internal/detail` directly under the module root), or move `main.go` into a directory under `lyrics/`.

Exercise 3 (Calculation): A module's `go.mod` contains the following:

```
module github.com/noahkahan/app
```

```
go 1.26
```

```
require (
    github.com/noahkahan/audio v1.4.0
    github.com/noahkahan/catalog v0.9.2
    golang.org/x/text v0.14.0 // indirect
)
```

```
github.com/noahkahan/audio v1.4.0 itself requires golang.org/x/text v0.12.0. github.com/noahkahan/catalog
v0.9.2 requires golang.org/x/text v0.14.0.
```

Under Go's Minimum Version Selection, which version of `golang.org/x/text` will the final build use? Explain why. Now suppose you add a new dependency that requires `golang.org/x/text v0.16.0`. What version will MVS select then?

First scenario: v0.14.0.

MVS collects the minimum required version from every module in the graph: - `github.com/noahkahan/app` itself requires `v0.14.0` (explicit `// indirect` entry). - `github.com/noahkahan/audio` requires `v0.12.0`. - `github.com/noahkahan/catalog` requires `v0.14.0`.

MVS takes the maximum of these minimums: $\max(v0.14.0, v0.12.0, v0.14.0) = v0.14.0$. The `// indirect` entry in the main module's `go.mod` already encodes this selection; `go mod tidy` placed it there when one of the direct dependencies required `v0.14.0` and the other only `v0.12.0`.

Second scenario: v0.16.0.

Adding a new dependency that requires `golang.org/x/text v0.16.0` raises the minimum for that module in the graph. MVS selects $\max(v0.14.0, v0.12.0, v0.14.0, v0.16.0) = v0.16.0$. After `go mod tidy`, the `// indirect` entry in `go.mod` is updated to `golang.org/x/text v0.16.0`. No other dependency's version changes.

Exercise 4 (Where is the bug?): The following module has this layout and code:

```
northernattitude/
  go.mod          (module github.com/noahkahan/northernattitude)
  main.go
  internal/
    config/
      config.go

player/main.go:

package main

import (
    "fmt"
    "github.com/noahkahan/northernattitude/internal/config"
)

func main() {
    fmt.Println(config.DefaultRegion)
}
```

What happens when you run `go build ./...` inside the `player/` module? Identify the bug and describe how to fix it without moving the `config` package out of `internal/`.

The build fails.

The `internal/` package belongs to the module `github.com/noahkahan/northernattitude`. The compiler's rule is that only code whose import path has `github.com/noahkahan/northernattitude` as a prefix may import packages under that module's `internal/`. The `player` module has path `github.com/noahkahan/player`, which does not share that prefix. The build error will be:

```
use of internal package github.com/noahkahan/northernattitude/internal/config not allowed
```

The fix — without moving `config` out of `internal/`:

The `config` package contains information that `northernattitude` treats as a private implementation detail. If `player` genuinely needs access to it, the right solution is for `northernattitude` to expose the data through a **public API**. Create an exported package, for example `github.com/noahkahan/northernattitude/region`, that wraps or re-exports the value from `internal/config`:

```
// northernattitude/region/region.go
package region

import "github.com/noahkahan/northernattitude/internal/config"

// DefaultRegion is the default geographic region.
var DefaultRegion = config.DefaultRegion
```

`player` then imports `github.com/noahkahan/northernattitude/region` instead of the internal package. The internal package remains private; its values are accessible only through the deliberately designed public surface.

Alternatively, if `player` and `northernattitude` are developed together and the restriction is inconvenient, use a Go workspace (`go work init ./northernattitude ./player`) and promote `config` to a shared module or to a non-internal path.

Exercise 5 (Write a program):

A complete implementation:

File `stickseason/go.mod`:

```
module github.com/noahkahan/stickseason
```

```
go 1.26
```

File `stickseason/tracks/tracks.go`:

```
package tracks

// Track holds the title and artist of a song.
type Track struct {
    Title string // song title
    Artist string // performing artist
}

// Catalog is the list of tracks in this module.
var Catalog = []Track{
    {Title: "Stick Season", Artist: "Noah Kahan"},
    {Title: "Northern Attitude", Artist: "Noah Kahan"},
}
```

File `stickseason/internal/format/format.go`:

```
package format

import (
    "fmt"
    "github.com/noahkahan/stickseason/tracks"
)

// Label returns a human-readable label for a track.
func Label(t tracks.Track) string {
    return fmt.Sprintf("%s by %s", t.Title, t.Artist)
}
```

File `stickseason/main.go`:

```
package main

import (
    "fmt"
    "github.com/noahkahan/stickseason/internal/format"
    "github.com/noahkahan/stickseason/tracks"
)

func main() {
    for _, t := range tracks.Catalog {
        fmt.Println(format.Label(t))
    }
}
```

Output:

```
Stick Season by Noah Kahan
Northern Attitude by Noah Kahan
```

Key observations:

- `main.go` can import `internal/format` because it is inside the same module (`github.com/noahkahan/stickseason`). An external module attempting the same import would receive a compile error.
 - `format.Label` is exported (capital L) so `main.go` can call it; it is still unreachable from outside the module because the package itself is under `internal/`.
 - The `tracks` package is public — any module that depends on `github.com/noahkahan/stickseason` could import it. Only `internal/format` is module-private.
-

Chapter 14: Essential Standard Library — Answers

Exercise 1 (Think about it): In Java, `InputStream`, `OutputStream`, `Reader`, and `Writer` are four separate abstract class hierarchies. Go has two interfaces — `io.Reader` and `io.Writer` — and a set of composition functions. What design decision makes Go’s two-interface model work where Java needed four base classes? What would be harder to express cleanly in Go’s model?

The key difference is that Java’s hierarchy distinguishes between **byte-oriented** I/O (`InputStream/OutputStream`) and **character-oriented** I/O (`Reader/Writer`), with four root types as a result. Go does not make that split at the interface level: `io.Reader` and `io.Writer` always deal in `[]byte`. Character encoding is handled separately — either at the edges (e.g., `bufio.Scanner` which returns `string` tokens), or by explicit conversion. This simplification is possible because Go treats `string` and `[]byte` as first-class, cheaply convertible types, so the language does not need a parallel hierarchy to make text feel natural.

The composition functions (`io.TeeReader`, `io.MultiWriter`, etc.) are ordinary functions that return an interface value. In Java the same decorators are abstract classes (`FilterInputStream`, `BufferedInputStream`) because the language needed a concrete supertype to share implementation; Go can express the same patterns with zero-allocation wrappers because interfaces are structural.

What is harder in Go’s model:

- **Seeking and positioning.** Java’s `RandomAccessFile` supports `seek` directly. Go separates this into `io.Seeker` (a third interface) and requires callers to do a type assertion or accept an `io.ReadSeeker` parameter.
 - **Buffered reads with unread/pushback.** Java’s `PushbackInputStream` is a first-class class. In Go you use `bufio.Reader.UnreadByte()` or `bufio.Reader.UnreadRune()`, which requires wrapping in `bufio` first.
 - **Encoding-aware text I/O.** Java’s `InputStreamReader` bridges bytes to characters with a named charset. In Go you must use third-party packages (e.g., `golang.org/x/text/encoding`) or write the conversion yourself.
-

Exercise 2 (What does this print?):

```
package main

import (
    "bufio"
    "fmt"
    "log/slog"
    "os"
    "strings"
    "time"
)

func main() {
```

```

logger := slog.New(slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{
    ReplaceAttr: func(groups []string, a slog.Attr) slog.Attr {
        if a.Key == slog.TimeKey {
            return slog.Attr{} // suppress the timestamp
        }
        return a
    },
}))

input := "Physical\nDon't Start Now\nPositions\n"
scanner := bufio.NewScanner(strings.NewReader(input))
count := 0
for scanner.Scan() {
    count++
}

logger.Info("scan complete",
    slog.Int("lines", count),
    slog.Duration("elapsed", 0*time.Millisecond),
)
}

```

Output:

```
level=INFO msg="scan complete" lines=3 elapsed=0s
```

Step-by-step:

1. A `slog.TextHandler` is created writing to `os.Stdout`. The `ReplaceAttr` function strips the `time` attribute, so no timestamp appears.
2. `strings.NewReader` wraps the literal string as an `io.Reader`. `bufio.NewScanner` wraps that reader.
3. The scanner splits on newlines (the default). The input has three non-empty lines ("Physical", "Don't Start Now", "Positions") followed by a trailing newline. `Scan` returns `true` three times and then `false` at EOF, so `count` ends up as 3.
4. `logger.Info` emits a text-format log line. The `time` key is suppressed by `ReplaceAttr`. `slog.Int("lines", 3)` formats as `lines=3`. `slog.Duration("elapsed", 0)` formats as `elapsed=0s` — `time.Duration` zero-value formats as "0s".

The exact key ordering in `log/slog` text format is: `level`, `msg`, then attributes in the order they were passed.

Exercise 3 (Calculation): You open a 10 MiB file and read it in three ways: (a) `os.ReadFile` into a `[]byte`, (b) `bufio.NewScanner` reading line by line, (c) `io.Copy(io.Discard, f)` using the default 32 KiB copy buffer. For each approach, estimate the peak heap allocation in MiB, assuming the file contains 100,000 lines of 100 bytes each. Which approach is best for counting lines without storing the content?

The file is 100,000 lines $\times$ 100 bytes = 10,000,000 bytes **9.5 MiB**.

(a) `os.ReadFile`

`os.ReadFile` reads the entire file into a single `[]byte`. Peak heap allocation: **9.5 MiB** (the whole file in one slice). Additionally, if you process the result into strings or split on newlines, you may double or triple the allocation. This is the simplest approach but the most memory-hungry for large files.

(b) `bufio.NewScanner`

`Scanner` uses an internal buffer (default 64 KiB maximum token size, starting at 4 KiB). It reads the file in chunks, scanning for newline boundaries. At any instant, only the current chunk plus the current token are

in memory. Peak heap allocation: **64 KiB** (the scanner's internal buffer) plus the length of the longest individual line. For 100-byte lines this is well under 1 MiB.

(c) `io.Copy(io.Discard, f)`

`io.Copy` uses a single 32 KiB stack-allocated copy buffer (it uses `*[32*1024]byte` internally; in practice this ends up on the heap due to escape analysis, but it is still a single fixed allocation). Peak heap allocation: **32 KiB**. However, this approach does not count lines — it just discards all bytes.

Best for counting lines without storing content: `bufio.NewScanner`.

`io.Copy(io.Discard, f)` uses the least memory but cannot count lines without inspecting the bytes. `bufio.NewScanner` counts lines with a constant-size buffer (< 1 MiB peak) and is the idiomatic Go choice. `os.ReadFile` uses the most memory and should be avoided for large files.

Exercise 4 (Where is the bug?):

```
package main

import (
    "fmt"
    "regexp"
)

func countMatches(texts []string, pattern string) int {
    total := 0
    for _, t := range texts {
        re := regexp.MustCompile(pattern)
        if re.MatchString(t) {
            total++
        }
    }
    return total
}

func main() {
    titles := []string{"positions", "Physical", "Don't Start Now", "thank u, next"}
    fmt.Println(countMatches(titles, `^[A-Z]`))
}
```

The bug: `regexp.MustCompile(pattern)` is called inside the `for` loop, so the pattern is compiled on every iteration. With four strings this is merely wasteful, but inside a hot path processing millions of records it becomes a serious performance problem — `regexp.MustCompile` parses the pattern, builds a finite automaton, and allocates memory each time.

The output is correct (it prints 2, matching "Physical" and "Don't Start Now"), so this is a **performance bug**, not a logic bug.

The fix: Compile the pattern once, before the loop. If the pattern is constant, hoist it to a package-level variable:

```
var startsUpperRE = regexp.MustCompile(`^[A-Z]`)

func countMatches(texts []string) int {
    total := 0
    for _, t := range texts {
        if startsUpperRE.MatchString(t) {
```

```

        total++
    }
}
return total
}

```

If the pattern is a runtime parameter, compile it once before the loop and return an error if the pattern is invalid:

```

func countMatches(texts []string, pattern string) (int, error) {
    re, err := regexp.Compile(pattern)
    if err != nil {
        return 0, fmt.Errorf("invalid pattern %q: %w", pattern, err)
    }
    total := 0
    for _, t := range texts {
        if re.MatchString(t) {
            total++
        }
    }
    return total, nil
}

```

Note the switch from `MustCompile` to `Compile` with a returned error — caller-supplied patterns should never `MustCompile` because a bad pattern would crash the program. `MustCompile` is reserved for compile-time-constant patterns where a bad pattern is a programmer error, not a user error.

Exercise 5 (Write a program):

```

package main

import (
    "flag"
    "fmt"
    "io/fs"
    "log/slog"
    "os"
    "path/filepath"
    "strings"
)

func main() {
    dir := flag.String("dir", ".", "directory to search")
    ext := flag.String("ext", ".go", "file extension to count")
    verbose := flag.Bool("verbose", false, "log each matching file")
    flag.Parse()

    logger := slog.New(slog.NewTextHandler(os.Stderr, nil))

    count := 0
    err := filepath.WalkDir(*dir, func(path string, d fs.DirEntry, err error) error {
        if err != nil {
            return err
        }
        if !d.IsDir() && strings.HasSuffix(d.Name(), *ext) {

```

```

        count++
        if *verbose {
            logger.Info("match", slog.String("file", path))
        }
    }
    return nil
})
if err != nil {
    logger.Error("walk failed", slog.Any("error", err))
    os.Exit(1)
}

fmt.Printf("found %d %s file(s) in %s\n", count, *ext, *dir)
}

```

Sample runs:

```

$ go run main.go -dir . -ext .go -verbose
time=... level=INFO msg=match file=main.go
found 1 .go file(s) in .

```

```

$ go run main.go -dir /usr/local/go/src -ext .go
found 1847 .go file(s) in /usr/local/go/src

```

Key points in the solution:

- `flag.Parse()` is called at the start of `main`, after all flag variables are defined, so all flags are parsed before use.
- `slog.New(slog.NewTextHandler(os.Stderr, nil))` writes structured logs to `stderr`, leaving `stdout` clean for program output.
- `filepath.WalkDir` is preferred over `filepath.Walk` because it passes `fs.DirEntry` (which avoids an extra `os.Stat` call per entry).
- `strings.HasSuffix(d.Name(), *ext)` matches only the file name, not the full path, so `--ext .go` does not accidentally match a directory named `foo.go/`.
- The error from `WalkDir` is checked and reported; a non-nil error from the callback halts the walk.

Chapter 15: JSON, HTTP, and the Web — Answers

Exercise 1 (Think about it): In Java with Spring MVC or JAX-RS, you annotate a class method with `@GetMapping("/songs/{id}")` or `@GET @Path("/songs/{id}")` and the framework discovers handlers via reflection and classpath scanning. In Go, you call `mux.HandleFunc("GET /songs/{id}/", getSong)` explicitly in `main`. What are the tradeoffs of each approach? Consider startup time, debuggability, IDE navigation, and what happens when two handlers are registered for the same pattern.

Annotation/reflection-based frameworks (Spring, JAX-RS):

- *Startup time*: The framework scans the classpath, processes annotations, and builds a routing table at startup. For large applications this can add seconds — sometimes tens of seconds. Spring Boot's startup time is a well-known pain point for serverless and container workloads.
- *IDE navigation*: IDEs understand Spring annotations deeply; `@GetMapping` provides clickable navigation to the handler. However, understanding the full request path often requires tracing through a chain of `@RequestMapping` annotations on the class, the method, and any inherited base classes.
- *Debuggability*: Routing bugs can be subtle; the framework discovers handlers at runtime, so a typo in a path annotation compiles cleanly and only fails when a request is made. Error messages from annotation-driven frameworks can be verbose and hard to relate back to specific source lines.

- *Duplicate pattern:* Spring raises a `BeanDefinitionOverrideException` or similar at startup.

Explicit registration (Go ServeMux):

- *Startup time:* Registration happens in `main` — it is just function calls. There is no scanning; startup overhead is negligible.
- *IDE navigation:* `mux.HandleFunc("GET /songs/{id}/", getSong)` — `getSong` is a direct function reference. Your IDE can jump to it with a single click, with no framework-specific plugin needed.
- *Debuggability:* The routing table is built from ordinary Go code. If you register the wrong path, you can add a `fmt.Println` or set a debugger breakpoint in `main` and see exactly what is registered.
- *Duplicate pattern:* Go 1.22 `ServeMux` panics at registration time if two patterns conflict. This is a startup crash rather than a silent routing bug, which is the right trade-off — it catches the mistake before any request is served.

The Go approach is more explicit and has less magic. The annotation approach provides more convenience in large teams where developers add handlers in many files and rely on the framework to assemble the routing table. Neither is universally better; the right choice depends on team size, application complexity, and how much framework overhead you are willing to accept.

Exercise 2 (What does this print?):

```
package main

import (
    "encoding/json"
    "fmt"
)

type Artist struct {
    Name      string `json:"name"`
    Country   string `json:"country,omitempty"`
    Secret    string `json:"-"`
}

func main() {
    a := Artist{Name: "Kali Uchis", Country: "", Secret: "Colombia"}
    data, _ := json.Marshal(a)
    fmt.Println(string(data))

    var b Artist
    json.Unmarshal([]byte(`{"name":"Rauw Alejandro","secret":"Puerto Rico"}`), &b)
    fmt.Printf("Name: %s, Secret: %q\n", b.Name, b.Secret)
}
```

Output:

```
{"name":"Kali Uchis"}
Name: Rauw Alejandro, Secret: ""
```

First Println: `a.Country` is `""`, which is the zero value for `string`. The tag `json:"country,omitempty"` causes `encoding/json` to omit the `country` field from the output. `a.Secret` is `"Colombia"`, but the tag `json:"-"` instructs the encoder to always skip this field regardless of its value. The result is `{"name":"Kali Uchis"}` — only `name` survives.

Second Printf: The JSON input contains a `"secret"` key. However, the Go struct has `Secret string` `json:"-"`. The `json:"-"` tag means `encoding/json` ignores this field during both marshalling and unmarshalling.

unmarshalling. The “secret” key in the JSON is silently discarded; b.Secret remains the zero value “”. b.Name is correctly set to “Rauw Alejandro” from the “name” key.

%q formats a string with Go double-quote syntax, so an empty string prints as “”.

Exercise 3 (Calculation): Consider the following `ServeMux` registration and the three incoming requests. For each request, state which handler function is called, or 404 if none matches.

```
mux := http.NewServeMux()
mux.HandleFunc("GET /tracks/", listTracks)
mux.HandleFunc("GET /tracks/{id}/", getTrack)
mux.HandleFunc("POST /tracks/", createTrack)
```

a. GET /tracks/ — `listTracks`

The request method is GET and the path is exactly /tracks/. The pattern GET /tracks/ is a subtree match that includes the exact path /tracks/. GET /tracks/{id}/ requires at least one additional path segment between the slashes (e.g., /tracks/42/), so it does not match /tracks/ alone. `listTracks` is called.

b. GET /tracks/42/ — `getTrack`

The request method is GET and the path is /tracks/42/. The pattern GET /tracks/{id}/ matches: {id} captures 42. `r.PathValue("id")` would return "42" inside the handler. `getTrack` is called.

c. DELETE /tracks/7/ — 404

None of the three registered patterns match a DELETE method on any path. GET /tracks/{id}/ matches the path shape but requires GET. `ServeMux` returns a 405 Method Not Allowed response in Go 1.22 when the path matches a pattern but the method does not. Effectively the caller receives an HTTP error response, not a call to any registered handler.

(Note: technically Go 1.22 `ServeMux` sends 405 Method Not Allowed with an `Allow` header listing valid methods when the path matches but the method does not — this is more precise than a plain 404.)

Exercise 4 (Where is the bug?):

```
func fetchLyrics(url string) (string, error) {
    resp, err := http.Get(url)
    if err != nil {
        return "", err
    }
    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return "", err
    }
    return string(body), nil
}
```

The bug: `resp.Body` is never closed.

When `http.Get` succeeds, `resp.Body` is a live network connection wrapped as an `io.ReadCloser`. If the function returns the body as a string but never calls `resp.Body.Close()`, the underlying TCP connection is not returned to the connection pool — it is leaked. Under load, a server making many requests will exhaust its file descriptors and connection pool, eventually causing all new HTTP requests to fail.

Note that the early-return error path `return "", err` after `io.ReadAll` also leaks the body.

The fix:

```

func fetchLyrics(url string) (string, error) {
    resp, err := http.Get(url)
    if err != nil {
        return "", err
    }
    defer resp.Body.Close() // close on any return, success or error

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return "", err
    }
    return string(body), nil
}

```

`defer resp.Body.Close()` immediately after the `err` check ensures the body is closed on every code path out of the function, including early returns. This is the canonical Go idiom for HTTP client response bodies.

Exercise 5 (Write a program):

```

package main

import (
    "encoding/json"
    "net/http"
    "strconv"
)

type Song struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Artist  string `json:"artist"`
}

var catalog = map[int]Song{
    1: {ID: 1, Title: "Todo De Ti",      Artist: "Rauw Alejandro"},
    2: {ID: 2, Title: "I Wish You Roses", Artist: "Kali Uchis"},
}

func listSongs(w http.ResponseWriter, r *http.Request) {
    songs := make([]Song, 0, len(catalog))
    for _, s := range catalog {
        songs = append(songs, s)
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(songs)
}

func getSong(w http.ResponseWriter, r *http.Request) {
    idStr := r.PathValue("id")
    id, err := strconv.Atoi(idStr)
    if err != nil {
        http.Error(w, "invalid id", http.StatusBadRequest)
        return
    }
}

```

```

    song, ok := catalog[id]
    if !ok {
        http.Error(w, "not found", http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(song)
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /songs/", listSongs)
    mux.HandleFunc("GET /songs/{id}/", getSong)
    http.ListenAndServe(":8080", mux)
}

```

Testing the server (with curl in a second terminal):

```

$ curl http://localhost:8080/songs/
[{"id":1,"title":"Todo De Ti","artist":"Rauw Alejandro"}, {"id":2,"title":"I Wish You Roses","artist":"K
$ curl http://localhost:8080/songs/1/
{"id":1,"title":"Todo De Ti","artist":"Rauw Alejandro"}

$ curl http://localhost:8080/songs/99/
not found

```

Key points illustrated by this solution:

- `json.NewEncoder(w).Encode(songs)` streams the JSON directly to the `http.ResponseWriter` without allocating an intermediate `[]byte`.
- `r.PathValue("id")` retrieves the wildcard captured by `{id}` in the Go 1.22 pattern.
- `strconv.Atoi` converts the string path segment to an integer; a malformed segment returns `400 Bad Request` rather than panicking.
- The `Content-Type` header is set before writing the body. Headers must be set before the first call to `Write` or `Encode` — once the body starts, the headers are sent and cannot be changed.
- The map iteration order in `listSongs` is random (Chapter 8). In a real service you would sort the result before encoding it to give clients a stable response.

Chapter 16: Database Access — Answers

Exercise 1 (Think about it): JDBC requires explicit transaction management and connection pooling through a `DataSource`, usually provided by an application server or a library like HikariCP. Go's `database/sql` builds connection pooling directly into `sql.DB`. What are the tradeoffs of each approach? In what situations might you still want an external connection pool in a Go application?

Go's approach is simpler for the common case: you call `sql.Open`, tune a few settings (`SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`), and the pool manages itself. There is no additional dependency, no configuration file, and no separate object to wire up. This is consistent with Go's philosophy of including batteries for common needs.

JDBC's reliance on an external pool (HikariCP, DBCP, c3p0, or an application server pool) adds setup complexity but provides more configurability. HikariCP, for example, offers connection validation queries, connection test-on-borrow, metric integration with Micrometer, and health check endpoints.

In a Go application you might still want an external or proxy pool in a few situations:

- **PgBouncer / ProxySQL:** These are database-side proxy pools that multiplex many application connections onto fewer server connections. They are useful when you have many application instances and the database itself limits total connections. `sql.DB`'s pool operates within one process; PgBouncer aggregates across many processes.
- **Serverless / short-lived processes:** If your Go binary starts and exits quickly (a CLI, a Lambda function), the in-process pool provides little benefit. A proxy pool keeps connections warm across many cold starts.
- **Observability:** Some proxy pools offer detailed query-level metrics and slow-query logging that are difficult to achieve from application code alone.

In most long-running Go services, the built-in pool is sufficient and external pooling adds unnecessary complexity.

Exercise 2 (What does this print?):

```
package main

import (
    "database/sql"
    "fmt"
)

func main() {
    a := sql.Null[string]{V: "Evergreen", Valid: true}
    b := sql.Null[string]{V: "Killing Me", Valid: false}
    c := sql.Null[int64]{V: 0, Valid: false}

    fmt.Println(a.Valid, a.V)
    fmt.Println(b.Valid, b.V)
    fmt.Println(c.Valid, c.V)
}
```

Output:

```
true Evergreen
false Killing Me
false 0
```

`sql.Null[T]` is a plain struct with two exported fields: `V` (the value) and `Valid` (a bool). It has no logic in its fields; they are whatever you set them to.

- `a` has `Valid: true` and `V: "Evergreen"`, so `fmt.Println` prints `true Evergreen`.
- `b` has `Valid: false` but `V` is still `"Killing Me"` — setting `Valid` to `false` does not zero out `V`. This might be surprising: the struct remembers the value even though it would represent `NULL` in the database. `fmt.Println` prints `false Killing Me`.
- `c` has `Valid: false` and `V: 0` (the zero value for `int64`). `fmt.Println` prints `false 0`.

The key takeaway: `Valid` controls whether the value is considered non-`NULL`; it does not affect what is stored in `V`. When scanning from a database, `Scan` sets `V` to the zero value and `Valid` to `false` for a `NULL` column.

Exercise 3 (Calculation): Trace the following transaction sequence.

```

tx, _ := db.BeginTx(ctx, nil)
defer tx.Rollback()

_, err := tx.ExecContext(ctx, "INSERT INTO songs (title, artist) VALUES (?, ?)",
    "On My Mama", "Victoria Monét")
if err != nil {
    return err
}

return tx.Commit()

```

Case A: ExecContext succeeds and Commit succeeds.

The deferred `tx.Rollback()` fires after `tx.Commit()` returns. Rollback on a committed transaction is a no-op — it returns `sql.ErrTxDone`, which is silently discarded because the return value of a deferred call is not used here. The row is **inserted and committed** permanently.

Case B: ExecContext succeeds but Commit returns an error.

`tx.Commit()` fails, so the function returns an error. The deferred `tx.Rollback()` then fires. However, when a commit fails at the database level, the transaction is typically already rolled back by the database. Rollback here is a safety net that confirms the abort. The row is **not inserted** — the transaction was not committed.

Case C: ExecContext returns an error.

The `if err != nil { return err }` branch fires, returning the error. The deferred `tx.Rollback()` fires before the function returns to the caller. The INSERT is undone (or was never applied, depending on the database). The row is **not inserted**.

In all three cases the deferred rollback provides a guarantee: the transaction is always cleaned up, regardless of which path the function takes. This is the entire point of the deferred rollback pattern.

Exercise 4 (Where is the bug?):

```

func getArtistSongs(ctx context.Context, db *sql.DB, artist string) ([]string, error) {
    rows, err := db.QueryContext(ctx,
        "SELECT title FROM songs WHERE artist = ?", artist)
    if err != nil {
        return nil, err
    }

    var titles []string
    for rows.Next() {
        var title string
        if err := rows.Scan(&title); err != nil {
            return nil, err
        }
        titles = append(titles, title)
    }
    return titles, nil
}

```

Two bugs:

Bug 1: rows is never closed. If `rows.Next()` completes the loop normally, `rows` is closed automatically. But if `rows.Scan` returns an error and the function returns early via `return nil, err`, `rows` is never closed. The connection borrowed from the pool is never returned, leaking it. Under sustained load this exhausts the pool.

Bug 2: rows.Err() is never checked. After the loop, rows.Err() may hold an error that caused iteration to stop early (e.g., a network failure mid-result-set). Ignoring it means the function silently returns a partial result as if it were complete.

Fixed version:

```
func getArtistSongs(ctx context.Context, db *sql.DB, artist string) ([]string, error) {
    rows, err := db.QueryContext(ctx,
        "SELECT title FROM songs WHERE artist = ?", artist)
    if err != nil {
        return nil, err
    }
    defer rows.Close() // always close, even on early return

    var titles []string
    for rows.Next() {
        var title string
        if err := rows.Scan(&title); err != nil {
            return nil, err
        }
        titles = append(titles, title)
    }
    if err := rows.Err(); err != nil { // check for iteration errors
        return nil, err
    }
    return titles, nil
}
```

defer rows.Close() immediately after checking the error from QueryContext is the standard pattern. Calling rows.Close() on already-closed rows is a no-op, so it is always safe.

Exercise 5 (Write a program):

```
package main

import (
    "context"
    "database/sql"
    "fmt"
    "log"

    _ "github.com/mattn/go-sqlite3"
)

type Playlist struct {
    ID          int
    Name        string
    Owner       string
    Description sql.Null[string]
}

func main() {
    db, err := sql.Open("sqlite3", ":memory:")
    if err != nil {
        log.Fatal(err)
    }
}
```

```

}
defer db.Close()

ctx := context.Background()

if err := db.PingContext(ctx); err != nil {
    log.Fatal(err)
}

_, err = db.ExecContext(ctx, `
    CREATE TABLE playlists (
        id            INTEGER PRIMARY KEY AUTOINCREMENT,
        name          TEXT      NOT NULL,
        owner         TEXT      NOT NULL,
        description TEXT
    )`)
if err != nil {
    log.Fatal(err)
}

// insert three rows inside a single transaction
if err := insertPlaylists(ctx, db); err != nil {
    log.Fatal(err)
}

// query and print all rows
rows, err := db.QueryContext(ctx,
    "SELECT id, name, owner, description FROM playlists ORDER BY id")
if err != nil {
    log.Fatal(err)
}
defer rows.Close()

for rows.Next() {
    var p Playlist
    if err := rows.Scan(&p.ID, &p.Name, &p.Owner, &p.Description); err != nil {
        log.Fatal(err)
    }
    desc := "(no description)"
    if p.Description.Valid {
        desc = p.Description.V
    }
    fmt.Printf("%d: %s by %s --- %s\n", p.ID, p.Name, p.Owner, desc)
}
if err := rows.Err(); err != nil {
    log.Fatal(err)
}
}

func insertPlaylists(ctx context.Context, db *sql.DB) error {
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
}

```

```

defer tx.Rollback() // no-op if Commit succeeds

stmt, err := tx.PrepareContext(ctx,
    "INSERT INTO playlists (name, owner, description) VALUES (?, ?, ?)")
if err != nil {
    return err
}
defer stmt.Close()

playlists := []Playlist{
    {
        Name:      "Victoria Vibes",
        Owner:      "coastin_fan",
        Description: sql.Null[string]{V: "Best of Victoria Monét", Valid: true},
    },
    {
        Name:      "Omar After Midnight",
        Owner:      "coastin_fan",
        Description: sql.Null[string]{V: "Evergreen on repeat", Valid: true},
    },
    {
        Name:      "Late Night Mix",
        Owner:      "apollo_stan",
        Description: sql.Null[string]{}, // NULL description
    },
}

for _, p := range playlists {
    if _, err := stmt.ExecContext(ctx, p.Name, p.Owner, p.Description); err != nil {
        return err
    }
}

return tx.Commit()
}

```

Output:

```

1: Victoria Vibes by coastin_fan --- Best of Victoria Monét
2: Omar After Midnight by coastin_fan --- Evergreen on repeat
3: Late Night Mix by apollo_stan --- (no description)

```

Key points demonstrated: - `sql.Open` + `PingContext` verifies connectivity before doing any work. - The transaction uses the **deferred rollback pattern**: `defer tx.Rollback()` is unconditional; `Commit` at the end makes it a no-op on success. - A prepared statement is created once inside the transaction and reused for each insert. - `sql.Null[string]` handles the nullable `description` column; `Valid: false` results in a `NULL` stored in the database and scanned back correctly. - `rows.Err()` is checked after the loop to catch any mid-stream errors.

Chapter 17: Generics — Answers

Exercise 1 (Think about it): Java generics use **type erasure**: at runtime, `List<String>` and `List<Integer>` are both just `List`. Generic type information is only available at compile time. Go generics use **monomor-**

phization (or a shared pointer-shaped representation): the compiler may generate distinct code for each instantiation. Describe one concrete advantage and one concrete disadvantage of each approach. How does type erasure affect what you can do with a Java generic type at runtime (e.g., `instanceof List<String>`)? Does Go have the same limitation?

Type erasure (Java):

Advantage: A single compiled class file handles all instantiations. `ArrayList<String>` and `ArrayList<Integer>` share bytecode, which keeps the compiled output compact and means that libraries compiled against an older JDK are forward-compatible with new generic code without recompilation.

Disadvantage: The generic type argument is gone at runtime. You cannot write `obj instanceof List<String>` — the JVM sees only `List`. Creating a generic array (`new T[]`) is illegal because the runtime cannot know the element type. Working around these limitations requires unchecked casts and `@SuppressWarnings("unchecked")`, which reintroduces the `ClassCastException` risk that generics were designed to prevent.

Monomorphization (Go):

Advantage: The compiler generates type-specific code, so operations on value types like `int` or `float64` are never boxed. A `Stack[int]` stores plain `int` values directly in the backing slice — no `Integer` wrapper objects, no extra allocations, no GC pressure. There is no unchecked cast at runtime; type safety is total.

Disadvantage: The compiler may produce multiple instantiations of the same function, increasing binary size. For large programs with many instantiation combinations, compile times can grow. (In practice, Go mitigates this by using a GC-shape-based approach that shares code for pointer-shaped types, but the tradeoff is still present for value types.)

Runtime reflection:

In Java, because of erasure, `List<String>.class` does not exist; you can only get `List.class`. `instanceof List<String>` is a compile-time warning and a runtime check against `List`, not `List<String>`. Accessing the actual type argument at runtime requires passing a `Class<T>` token explicitly.

In Go, you can use `reflect.TypeOf` to inspect the concrete type of a value, and since Go does not erase type information the way Java does, the concrete type is always available. However, Go reflection operates on concrete values, not on type parameters themselves — you cannot query “what was `T`?” from inside a generic function without passing the type explicitly. Both languages have some limitations here, but the flavor of the limitation differs.

Exercise 2 (What does this print?):

```
package main

import "fmt"

func Filter[T any](s []T, keep func(T) bool) []T {
    var out []T
    for _, v := range s {
        if keep(v) {
            out = append(out, v)
        }
    }
    return out
}

type BPM int
```

```
func main() {
    beats := []BPM{72, 128, 96, 140, 80}
    fast := Filter(beats, func(b BPM) bool { return b >= 120 })
    fmt.Println(fast)

    words := []string{"greedy", "Heather", "Astronomy", "you broke me first"}
    long := Filter(words, func(s string) bool { return len(s) > 7 })
    fmt.Println(long)
}
```

Output:

```
[128 140]
[Astronomy you broke me first]
```

For the first call, `T` is inferred as `BPM`. The predicate keeps elements greater than or equal to 120. 72, 96, and 80 are below 120 and are excluded. 128 and 140 pass and are appended to `out`.

For the second call, `T` is inferred as `string`. The predicate keeps strings longer than 7 characters. "greedy" has 6 characters (excluded), "Heather" has 7 (excluded, because 7 is not greater than 7), "Astronomy" has 9 (included), and "you broke me first" has 18 (included).

Exercise 3 (Calculation): The function below has the signature `func Reduce[T, U any](s []T, init U, f func(U, T) U) U`. Trace the execution of `Reduce([]int{1, 2, 3, 4}, 0, func(acc, v int) int { return acc + v })`. What is the concrete type bound to `T`? What is the concrete type bound to `U`? What value does the function return, and what are the intermediate values of `acc` after each call to `f`?

`T` is bound to `int` (the element type of `[]int{1, 2, 3, 4}`). `U` is also bound to `int` (the type of the initial accumulator 0 and the return type of `f`).

The function body would be:

```
func Reduce[T, U any](s []T, init U, f func(U, T) U) U {
    acc := init
    for _, v := range s {
        acc = f(acc, v)
    }
    return acc
}
```

Tracing each call to `f`:

Iteration	acc before	v	acc after (acc + v)
1	0	1	1
2	1	2	3
3	3	3	6
4	6	4	10

The function returns **10**.

Exercise 4 (Where is the bug?):

```
package main

import "fmt"
```

```

type Playlist []string

func Dedupe[T any](s []T) []T {
    seen := make(map[T]bool)
    var out []T
    for _, v := range s {
        if !seen[v] {
            seen[v] = true
            out = append(out, v)
        }
    }
    return out
}

func main() {
    p := Playlist{"greedy", "Heather", "greedy", "Astronomy", "Heather"}
    fmt.Println(Dedupe(p))
}

```

The bug: T is constrained to any, but map[T]bool requires T to be comparable. The compiler rejects this with an error like:

```
invalid map key type T (missing comparable constraint)
```

Using a type as a map key requires that it support == and !=. any does not guarantee this.

The fix: Change the constraint from any to comparable:

```

func Dedupe[T comparable](s []T) []T {
    seen := make(map[T]bool)
    var out []T
    for _, v := range s {
        if !seen[v] {
            seen[v] = true
            out = append(out, v)
        }
    }
    return out
}

```

string and Playlist (whose underlying type is []string) — wait: Playlist is []string, and slices are **not** comparable. So even with comparable, passing Playlist would fail because []string does not satisfy comparable.

The call in main passes p (of type Playlist, underlying type []string) directly. Slices are never comparable in Go.

The correct fix is to change the call to pass the string slice elements rather than the slice type:

```

func main() {
    p := []string{"greedy", "Heather", "greedy", "Astronomy", "Heather"}
    fmt.Println(Dedupe(p))
}

```

With T comparable and p as []string, the output is:

```
[greedy Heather Astronomy]
```

In summary, there are two bugs: the constraint must be `comparable`, and `Playlist` (a `[]string`) cannot be used as a map key because slices are not comparable.

Exercise 5 (Write a program):

```
package main

import "fmt"

// Set is a generic unordered collection of unique comparable values.
type Set[T comparable] struct {
    m map[T]struct{}
}

// Add inserts v into the set.
func (s *Set[T]) Add(v T) {
    if s.m == nil {
        s.m = make(map[T]struct{}) // lazy initialization
    }
    s.m[v] = struct{}{}
}

// Contains reports whether v is in the set.
func (s *Set[T]) Contains(v T) bool {
    _, ok := s.m[v]
    return ok
}

// Values returns all elements of the set as a slice in unspecified order.
func (s *Set[T]) Values() []T {
    out := make([]T, 0, len(s.m))
    for v := range s.m {
        out = append(out, v) // iteration order is random
    }
    return out
}

func main() {
    var songs Set[string]
    songs.Add("greedy")
    songs.Add("you broke me first")
    songs.Add("Heather")
    songs.Add("Astronomy")
    songs.Add("Heather") // duplicate --- should be ignored

    fmt.Println("length:", len(songs.Values())) // 4
    fmt.Println("contains Heather:", songs.Contains("Heather")) // true
    fmt.Println("contains Levitating:", songs.Contains("Levitating")) // false
}
```

Output:

```
length: 4
contains Heather: true
```

contains Levitating: false

`map[T]struct{}` is the standard Go idiom for a set. An empty struct (`struct{}`) occupies zero bytes, so only the keys consume memory. The second `Add("Heather")` call is a no-op because the map key already exists — map assignment is idempotent. `Values()` returns four strings because the duplicate was silently dropped, but their order will vary between runs since Go map iteration is randomized.

Chapter 18: Testing — Answers

Exercise 1 (Think about it): JUnit 5's `@ParameterizedTest` with `@CsvSource` and Go's table-driven tests with `t.Run` both let you run the same logic against many inputs. Describe two concrete advantages that Go's table-driven approach gives you over `@CsvSource`. Then explain the key behavioral difference between `t.Fatal` and `t.Error` inside a subtest, and describe a scenario where you would deliberately choose `t.Error` over `t.Fatal`.

Table-driven tests vs `@CsvSource` — two concrete advantages:

1. **Structured, type-safe test cases.** With `@CsvSource`, each row is a comma-separated string; numeric values must be parsed at runtime and type errors surface only when the test runs. A Go table is a slice of structs — the compiler checks every field at compile time. If you rename a field or change its type, the build breaks immediately. There is no equivalent compile-time safety in `@CsvSource`.
2. **Arbitrary Go values in each case.** `@CsvSource` can only express types that JUnit knows how to convert from strings: primitives, strings, enums. A Go table can hold any value — a function, an `error`, a struct, a slice — as a field in the test case struct. This lets you express cases like “given this pre-built request object, expect this error” without any serialization or custom converter.

`t.Fatal` vs `t.Error` inside a subtest:

Inside a `t.Run` subtest, `t.Fatal` stops only that subtest's goroutine — it calls `runtime.Goexit()` on the subtest's goroutine. The outer test loop continues and the next subtest runs normally. `t.Error` also affects only the subtest: it marks it as failed but the subtest continues executing.

A scenario where you would choose `t.Error` over `t.Fatal`: when you are validating multiple independent fields of a response struct and you want to see all failures at once. For example, if you call an HTTP handler and want to check both the status code and the response body, use `t.Error` for each. If you used `t.Fatal` on the status code check, a wrong status code would hide a potentially wrong body — you would have to fix and re-run to see the body error. With `t.Error`, one failing run shows you everything that is wrong.

Exercise 2 (What does this print?): Trace the output when this test is run with `go test -v`.

```
package music_test

import "testing"

func checkPositive(t *testing.T, n int) {
    if n <= 0 {
        t.Errorf("expected positive, got %d", n)
    }
}

func TestGolden(t *testing.T) {
    checkPositive(t, 1)
    t.Log("checked 1")
    checkPositive(t, -1)
```

```

    t.Log("checked -1")
    checkPositive(t, 2)
    t.Log("checked 2")
}

```

Output (with `go test -v`):

```

=== RUN    TestGolden
    music_test.go:7: expected positive, got -1
    music_test.go:11: checked 1
    music_test.go:13: checked -1
    music_test.go:15: checked 2
--- FAIL: TestGolden (0.00s)
FAIL

```

Key points to trace:

- `checkPositive(t, 1)`: $1 > 0$, so no error is recorded.
- `t.Log("checked 1")`: message is queued.
- `checkPositive(t, -1)`: $-1 \leq 0$, so `t.Errorf` is called. `t.Errorf` is `t.Error` with formatting — it records a failure message and marks the test failed, but **does not stop execution**. The test keeps running.
- `t.Log("checked -1")`: message is queued.
- `checkPositive(t, 2)`: $2 > 0$, no error.
- `t.Log("checked 2")`: message is queued.
- Because the test is marked failed, all `t.Log` output is printed (with `-v`, log output is always printed regardless of pass/fail).

The test finishes with `--- FAIL`. Execution continues past the failing check because `t.Errorf` is used, not `t.Fatalf`.

Note on `t.Helper()` absence: `checkPositive` does not call `t.Helper()`. As a result, the failure line reported is inside `checkPositive` (the `t.Errorf` call), not in `TestGolden` where `checkPositive(-1)` was called. Adding `t.Helper()` as the first line of `checkPositive` would make the reported line point to `checkPositive(t, -1)` in `TestGolden` instead.

Exercise 3 (Calculation): A benchmark function has the following structure:

```

func BenchmarkWoman(b *testing.B) {
    for range b.N {
        _ = processTrack("Woman")
    }
}

```

On the first probe the framework sets `b.N = 1` and measures elapsed time. It then sets `b.N = 100`, then `b.N = 10_000`, then `b.N = 1_000_000`. The framework stops when the total elapsed time exceeds one second. If `processTrack` takes exactly 500 ns per call, at which value of `b.N` does the total elapsed time first exceed one second? What is the reported ns/op value?

Answer:

Total elapsed time = `b.N × 500 ns`.

b.N	Total time
1	500 ns
100	50,000 ns = 50 µs
10,000	5,000,000 ns = 5 ms

b.N	Total time
1,000,000	500,000,000 ns = 500 ms

None of those values exceeds one second. The framework continues increasing `b.N`. The next typical value after 1,000,000 is 2,000,000:

b.N	Total time
2,000,000	1,000,000,000 ns = 1 s

At `b.N = 2,000,000` the total time is exactly 1 second, which meets (ties) the threshold.

b.N = 2,000,000 is where the run stops (or the next step after, depending on exact rounding in the real framework).

Reported ns/op: The framework reports `total_time / b.N = 1,000,000,000 ns / 2,000,000 = 500 ns/op`.

This matches `processTrack`'s actual per-call cost — the benchmark is accurate.

Exercise 4 (Where is the bug?): The following test helper is supposed to make failure output point to the call site in `TestAboutDamnTime`, but it does not. Identify the bug and show the fix.

```
package music_test

import "testing"

func assertNormalized(t *testing.T, input, want string) {
    got := normalize(input)
    if got != want {
        t.Fatalf("normalize(%q): got %q, want %q", input, got, want)
    }
}

func TestAboutDamnTime(t *testing.T) {
    assertNormalized(t, "about damn time", "About Damn Time")
    assertNormalized(t, "good as hell", "Good as Hell")
}
```

The bug: `assertNormalized` does not call `t.Helper()`.

When `t.Fatalf` fires inside `assertNormalized`, Go's test framework records the file and line number of the `t.Fatalf` call inside the helper. The reported failure location is something like:

```
music_test.go:8: normalize("about damn time"): got "about Damn Time", want "About Damn Time"
```

That points inside `assertNormalized`, not to the line in `TestAboutDamnTime` that triggered the failure. You have to manually trace back to find which call site is responsible.

The fix: add `t.Helper()` as the first statement in `assertNormalized`:

```
func assertNormalized(t *testing.T, input, want string) {
    t.Helper() // attribute failures to the caller, not this function
    got := normalize(input)
    if got != want {
        t.Fatalf("normalize(%q): got %q, want %q", input, got, want)
    }
}
```

```
    }
}
```

With `t.Helper()` present, the reported failure location becomes:

```
music_test.go:13: normalize("about damn time"): got "about Damn Time", want "About Damn Time"
```

That line number points directly to `assertNormalized(t, "about damn time", "About Damn Time")` in `TestAboutDamnTime`, which is exactly where the problematic call lives.

Secondary note: `t.Fatalf` inside a helper is fine when subsequent checks in the same test function would be meaningless if this check fails. Here, if `normalize("about damn time")` returns a wrong value the second check can still run independently, so `t.Errorf` could be argued as a better choice — but whether to use `Fatal` or `Error` is a judgment call; the `t.Helper()` omission is the clear bug.

Exercise 5 (Write a program): Write a table-driven test for `TitleCase`. Your test must include at least five cases covering normal input, empty string, all-caps input, and a multi-word title. Use `t.Run` for each case and `t.Helper` in any helper you write.

```
package music_test

import (
    "strings"
    "testing"
    "unicode"
)

// TitleCase converts a string to title case.
// Each word's first letter is uppercased; the rest are lowercased.
// Words are separated by spaces.
func TitleCase(s string) string {
    words := strings.Fields(s)
    for i, w := range words {
        if len(w) == 0 {
            continue
        }
        runes := []rune(w)
        runes[0] = unicode.ToUpper(runes[0])
        for j := 1; j < len(runes); j++ {
            runes[j] = unicode.ToLower(runes[j])
        }
        words[i] = string(runes)
    }
    return strings.Join(words, " ")
}

// assertEqual is a helper that reports mismatches at the caller's site.
func assertEqual(t *testing.T, got, want, label string) {
    t.Helper()
    if got != want {
        t.Errorf("%s: got %q, want %q", label, got, want)
    }
}

func TestTitleCase(t *testing.T) {
```



```

cases := []struct {
    name  string
    input string
    want  string
}{
    {name: "empty",      input: "",      want: ""},
    {name: "single word", input: "golden",  want: "Golden"},
    {name: "multi-word", input: "good as hell", want: "Good As Hell"},
    {name: "all caps",   input: "ABOUT DAMN TIME", want: "About Damn Time"},
    {name: "mixed case", input: "wOmAn",    want: "Woman"},
    {name: "already title", input: "Good As Hell", want: "Good As Hell"},
}

for _, tc := range cases {
    t.Run(tc.name, func(t *testing.T) {
        got := TitleCase(tc.input)
        assertEquals(t, got, tc.want, "TitleCase("+tc.input+")")
    })
}

```

Running `go test -v` produces:

```

=== RUN   TestTitleCase
=== RUN   TestTitleCase/empty
=== RUN   TestTitleCase/single_word
=== RUN   TestTitleCase/multi-word
=== RUN   TestTitleCase/all_caps
=== RUN   TestTitleCase/mixed_case
=== RUN   TestTitleCase/already_title
--- PASS: TestTitleCase (0.00s)
    --- PASS: TestTitleCase/empty (0.00s)
    --- PASS: TestTitleCase/single_word (0.00s)
    --- PASS: TestTitleCase/multi-word (0.00s)
    --- PASS: TestTitleCase/all_caps (0.00s)
    --- PASS: TestTitleCase/mixed_case (0.00s)
    --- PASS: TestTitleCase/already_title (0.00s)

```

PASS

Notes on the solution:

- The `name` field in each case struct is passed to `t.Run`, producing descriptive subtest names in the output. A failing case shows up as `--- FAIL: TestTitleCase/all_caps` rather than just `--- FAIL: TestTitleCase`.
- `assertEquals` calls `t.Helper()` so that any failure message points to the line inside the `t.Run` body that called `assertEquals`, not to the `t.Errorf` line inside `assertEquals` itself.
- `t.Errorf` (not `t.Fatalf`) is used in the helper because each subtest has only one assertion; there is no reason to stop early. If the helper checked multiple things, `t.Fatalf` could be appropriate for a critical precondition.
- The six cases satisfy the problem requirements: empty string, single word (normal), multi-word, all-caps, and mixed case. The “already title” case is a bonus regression check.